

TEXTURE MAPPING

Lecturer: Dr. Ufuk Çelikcan

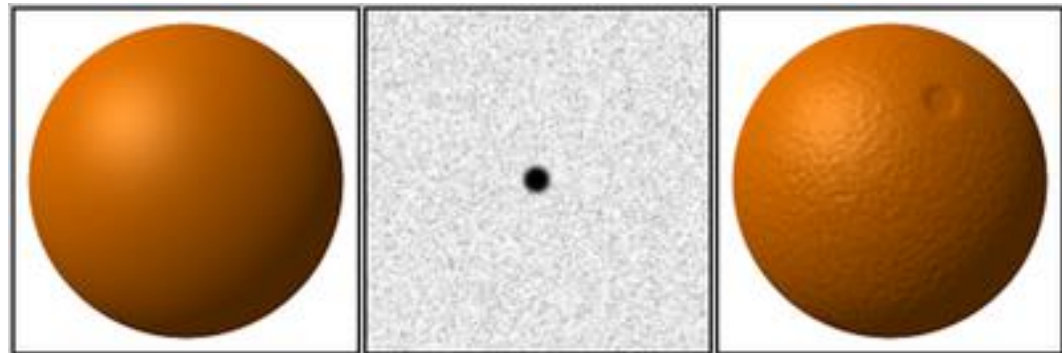
Based on the slides by: E. Angel and D. Shreiner

Modeling an Orange

- Consider the problem of modeling an orange (the fruit)
- Start with an orange-colored sphere?
 - Too simple
- Replace sphere with a more complex shape?
 - Very hard to properly capture surface characteristics (small dimples) realistically
 - Takes too many polygons to model all the dimples

Modeling an Orange (2)

- Take a picture of a real orange, scan it, and “paste” onto simple geometric model?
 - This process is known as **Texture Mapping**
 - Still might not be sufficient though, because the resulting surface will still be smooth
 - Need to change local shape during shading
- + Bump Mapping**

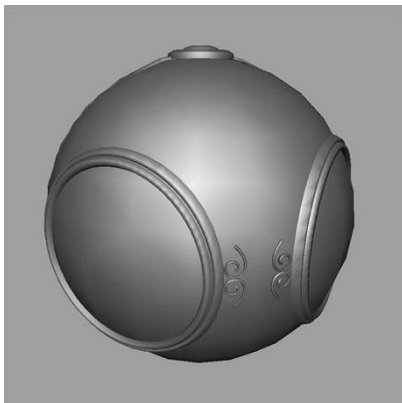


A texture can represent..

- Color
- Specularity (patches of shininess)
- Transparency (patches of clearness)
- Normal vector changes (bump maps)
- Reflected light (environment maps)
- Shadows
- Change in surface height (displacement maps)

Types of Mapping

- Texture Mapping
 - Uses images to fill inside of polygons
- Environment (reflection mapping)
 - Uses a capture of the surrounding environment for texture maps
 - Allows simulation of highly specular surfaces
- Bump mapping
 - Emulates altering normal vectors during the rendering process



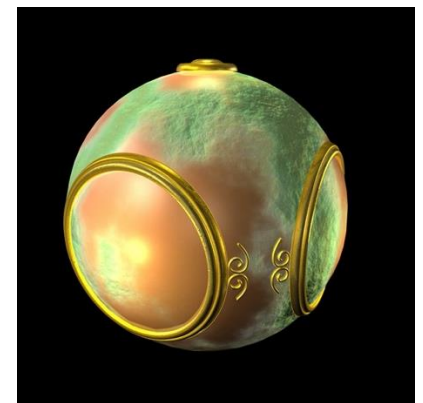
geometric model



texture mapped



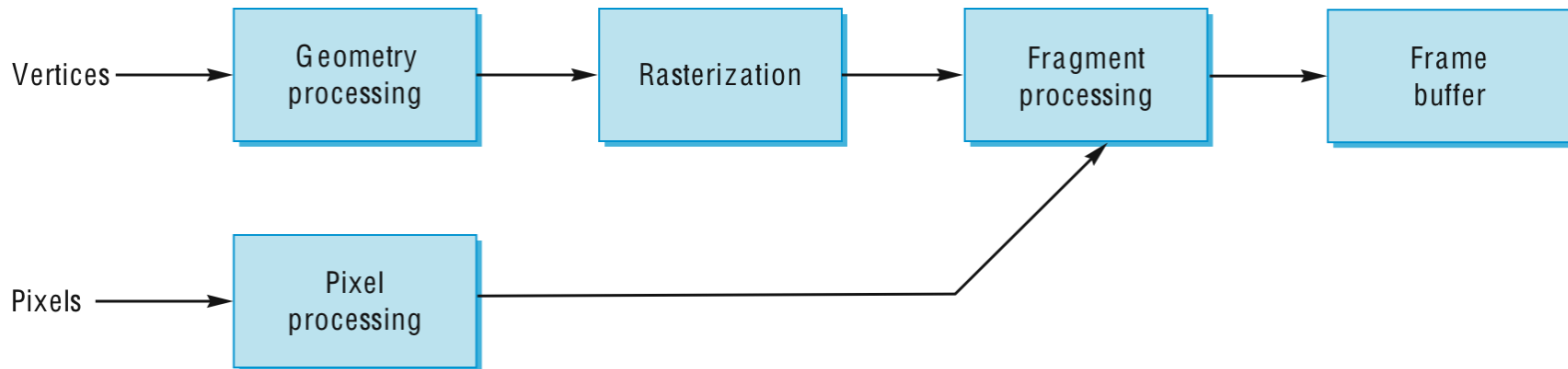
environment
mapped



texture mapped

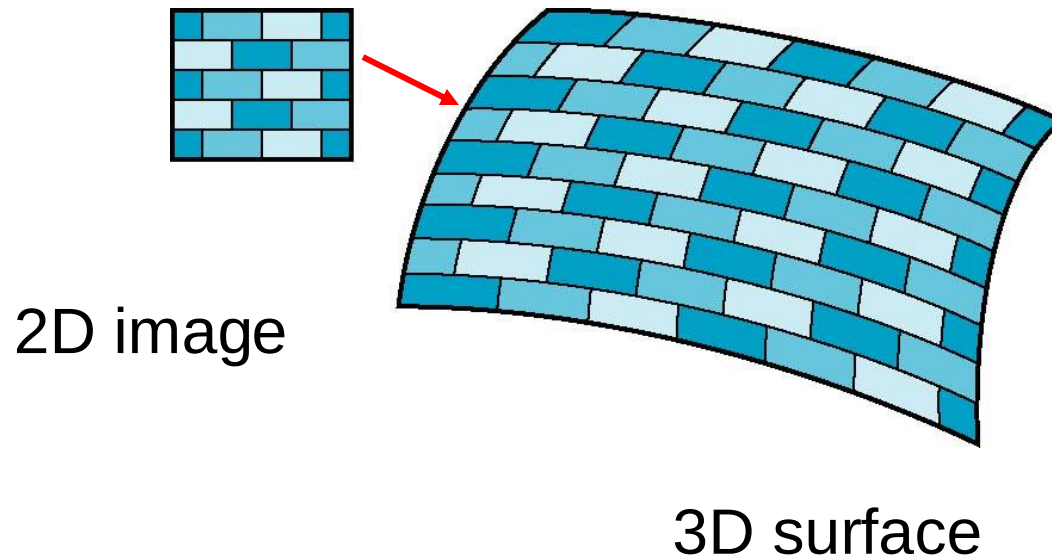
Where does mapping take place?

- Mapping techniques are implemented **at the end of the rendering pipeline**
 - Very efficient because fewer polygons make it past the clipper



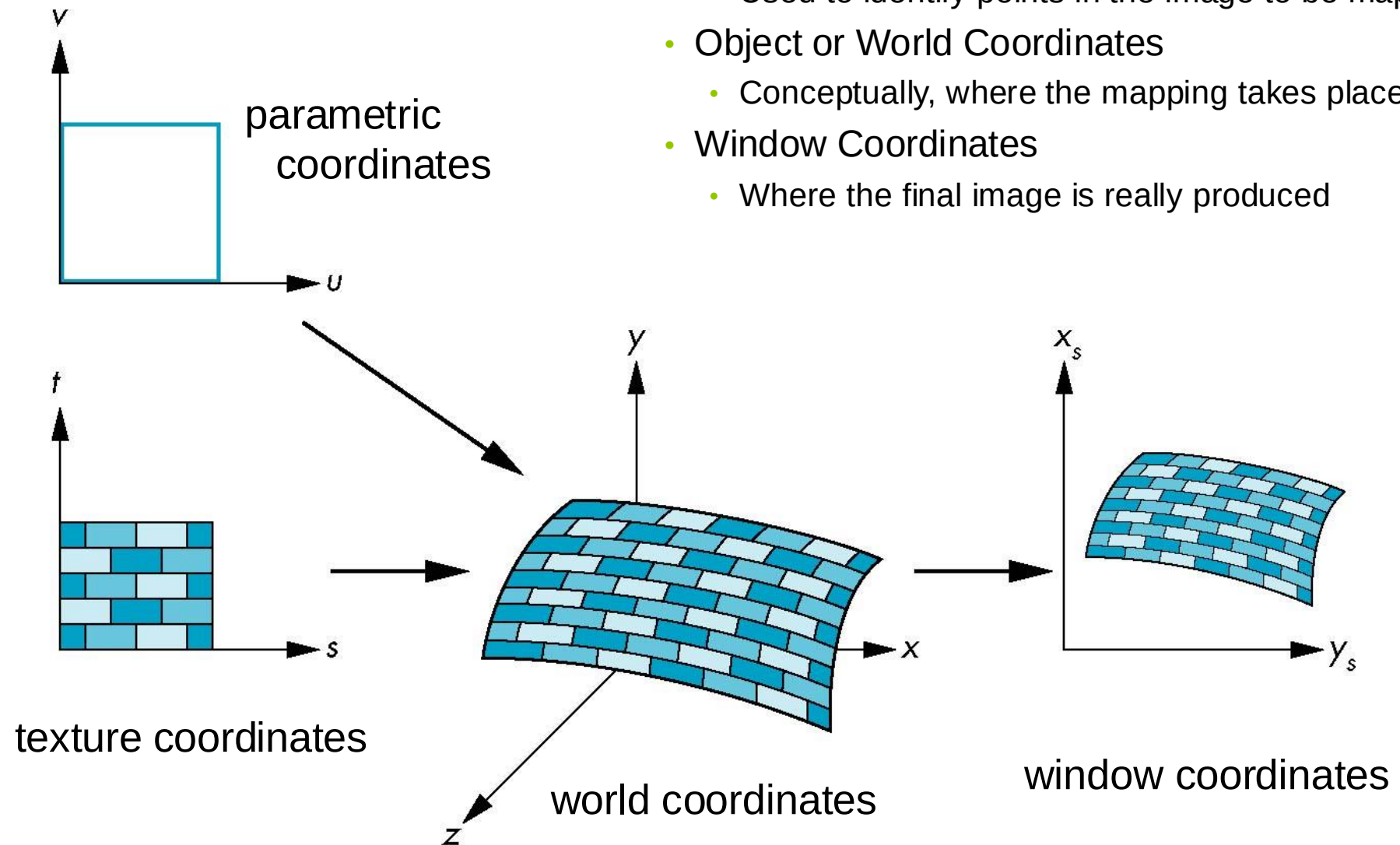
Is texture mapping simple?

- Although the idea is simple: map an image to a surface
- there are 3 or 4 coordinate systems involved



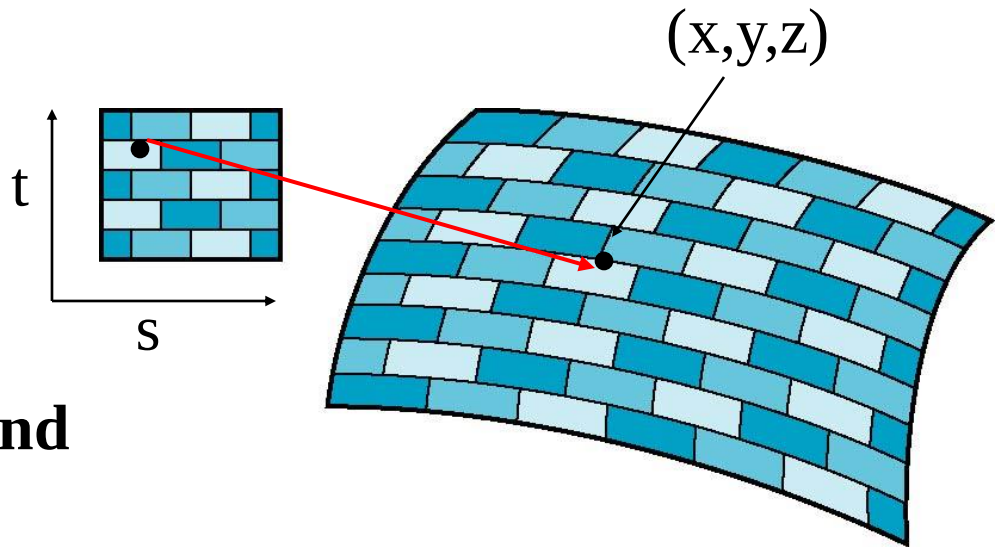
Coordinate Systems

- Parametric coordinates
 - Used to model intermediary curves and surfaces
- Texture coordinates
 - Used to identify points in the image to be mapped
- Object or World Coordinates
 - Conceptually, where the mapping takes place
- Window Coordinates
 - Where the final image is really produced



Mapping Functions

- Basic problem is **how to find the mappings**
- Consider mapping from texture coordinates to a point on a surface: “Given a texture pixel, which point over the object surface does it cover?”
- Needs three functions
$$x = x(s,t)$$
$$y = y(s,t)$$
$$z = z(s,t)$$
- **But we actually want to go the other way around**

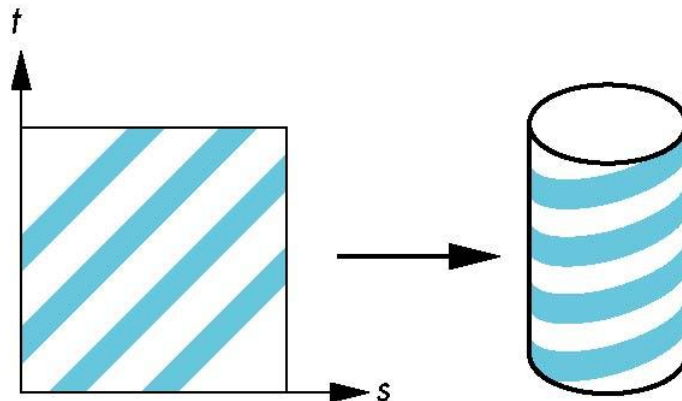


Backward Mapping

- **Given a point on an object, we want to know which point in the texture it corresponds to**
- Need a map of the form
$$s = s(x,y,z)$$
$$t = t(x,y,z)$$
- Such functions are difficult to find in general

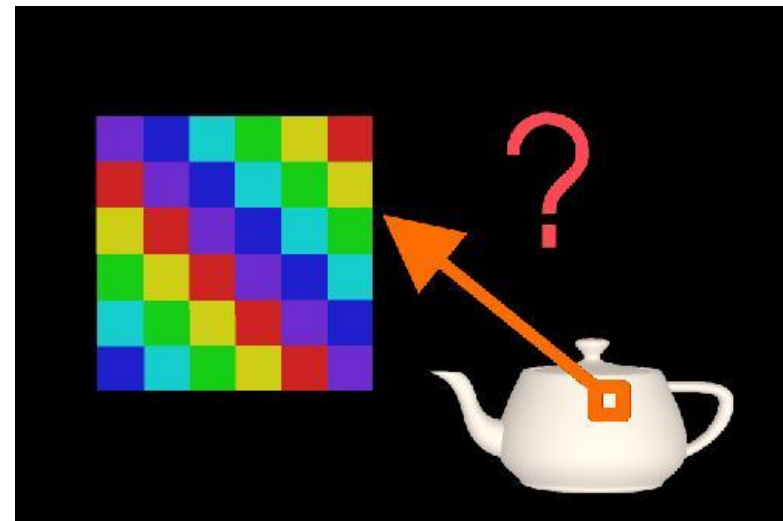
Two-part mapping

- One solution to the mapping problem is :
 1. first map the texture to a simpler **intermediate surface**
 2. then map from intermediate surface to the actual target surface
- Example: map to a cylinder



Texturing: Parameterization (Object Mesh)

- How do we assign texture coordinates to objects?
 - Problem: Map from 3D to 2D
 - Idea: Map (x, y, z) to an intermediate space (u, v)
 - **Projector** function to obtain intermediary surface coordinates (u, v)
 - **Corresponder** function to find **texel** coordinates (s, t)
- Filter texel at (s, t)
 - Modify pixel (i, j)

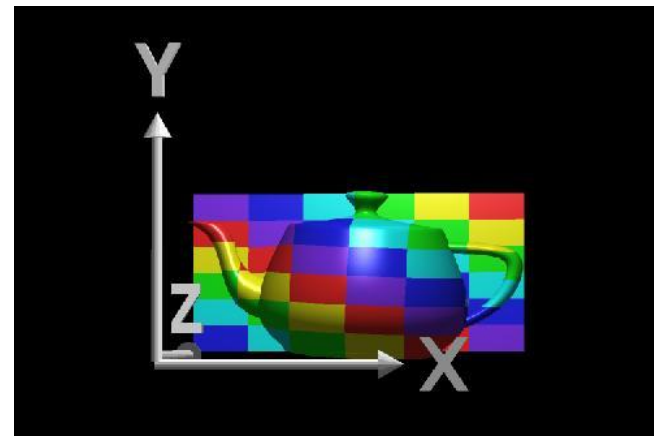


courtesy of R. Wolfe

Rasterization

Projector Functions

- How do we map the texture onto a arbitrary (complex) object?
 - Construct a mapping between the 3-D point to an intermediate surface
 - Why?
 - The intermediate surface is simple $\Rightarrow$ we know its characteristics
 - Still a 3D surface, but easier to map to texture space (2D)
 - Easy to parameterize the intermediate surface in 2D, i.e. (u, v) space
 - Idea: Project each object point to the intermediate surface with a parallel or perspective projection
 - The focal point is usually placed inside the object
- Plane
 - Cylinder
 - Sphere
 - Cube
 - Mesh: piece-wise planar



courtesy of R. Wolfe

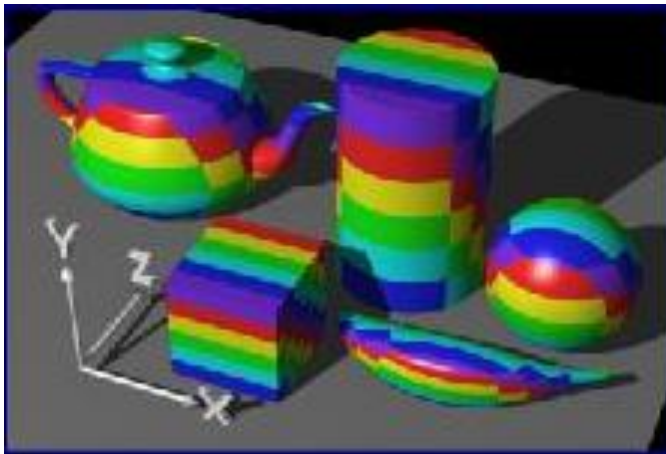
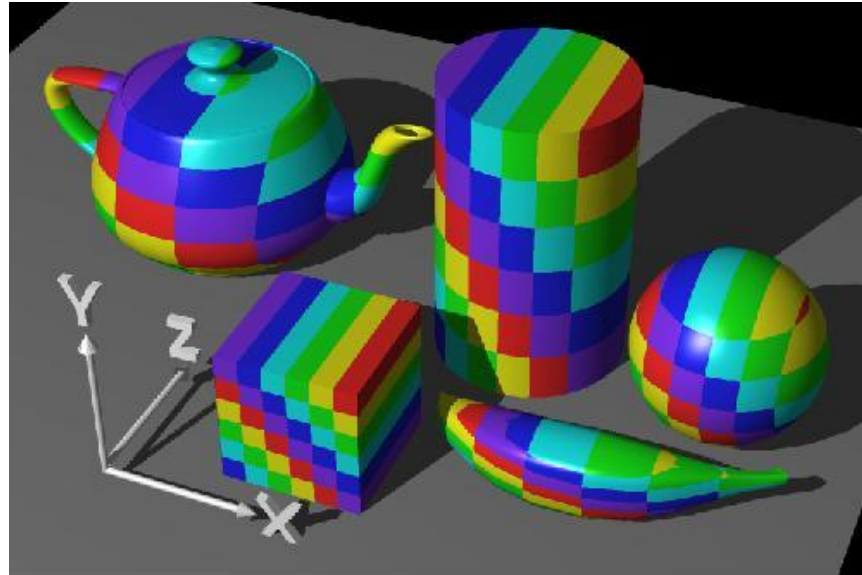
Planar projector

Planar Projector

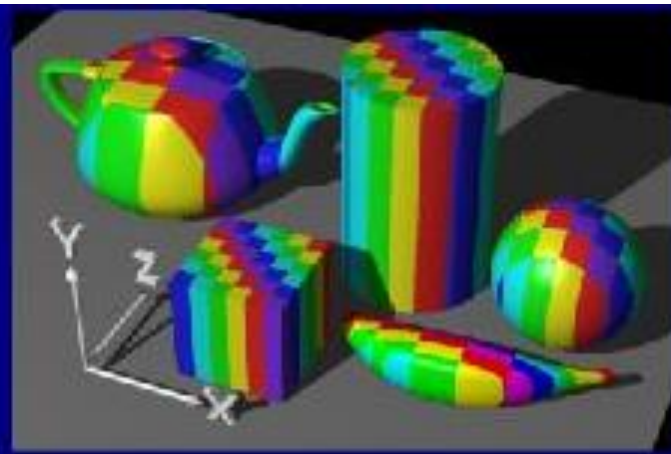
Orthographic projection
onto XY plane:

$$u = x, \quad v = y$$

Corresponder function is simply $s = u$
 $s, t, u, v \in [0,1]$ $t = v$
maps from texture space to the rectangle



...onto YZ plane



...onto XZ plane

courtesy of
R. Wolfe

Cylindrical Mapping

parametric cylinder

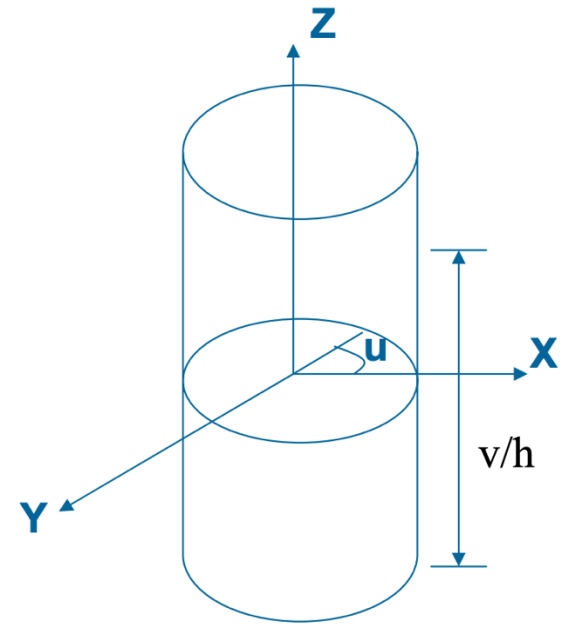
$$x = r \cos(2\pi u)$$

$$y = r \sin(2\pi u)$$

$$z = v/h$$

Projector function

$$u, v \in [0,1]$$



maps

- from a rectangle in **u,v** space
- to a cylinder of radius **r** and height **h** in world coordinates

$$s = u$$

$$t = v$$

Corresponder function

$$s, t, u, v \in [0,1]$$

maps from texture space to the rectangle

Cylindrical Projector



courtesy of
R. Wolfe

Spherical Map

In a similar way, we can use a parametric sphere such as:

$$x = r \cos(2\pi u)$$

$$y = r \sin(2\pi u) \cos(2\pi v)$$

$$z = r \sin(2\pi u) \sin(2\pi v)$$

Projector function

$$s, t, u, v \in [0, 1]$$

[u,v: one is the vertical angle and the other is the horizontal angle but the sphere on the right has a different parameterization]

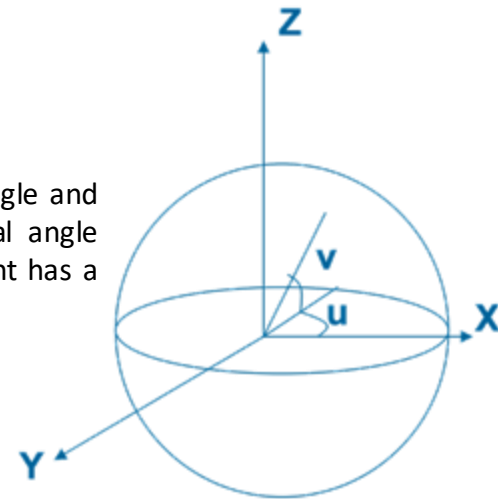
Then once again

$$s = u$$

$$t = v$$

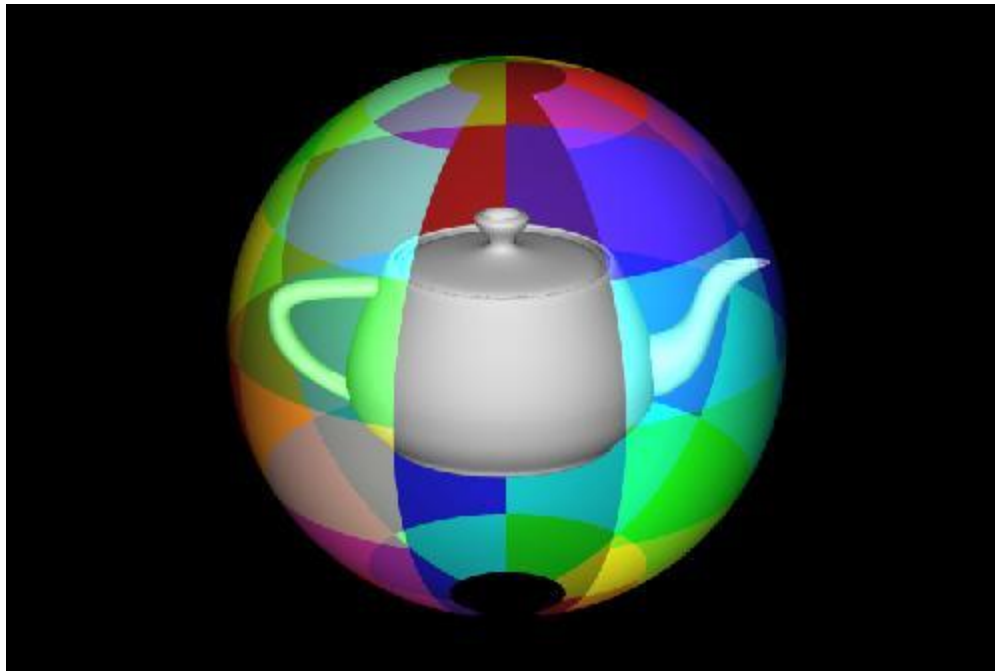
Corresponder function

maps from texture space to the rectangle



- but have to decide where to put the distortion.
- **Spheres are used in environmental maps**

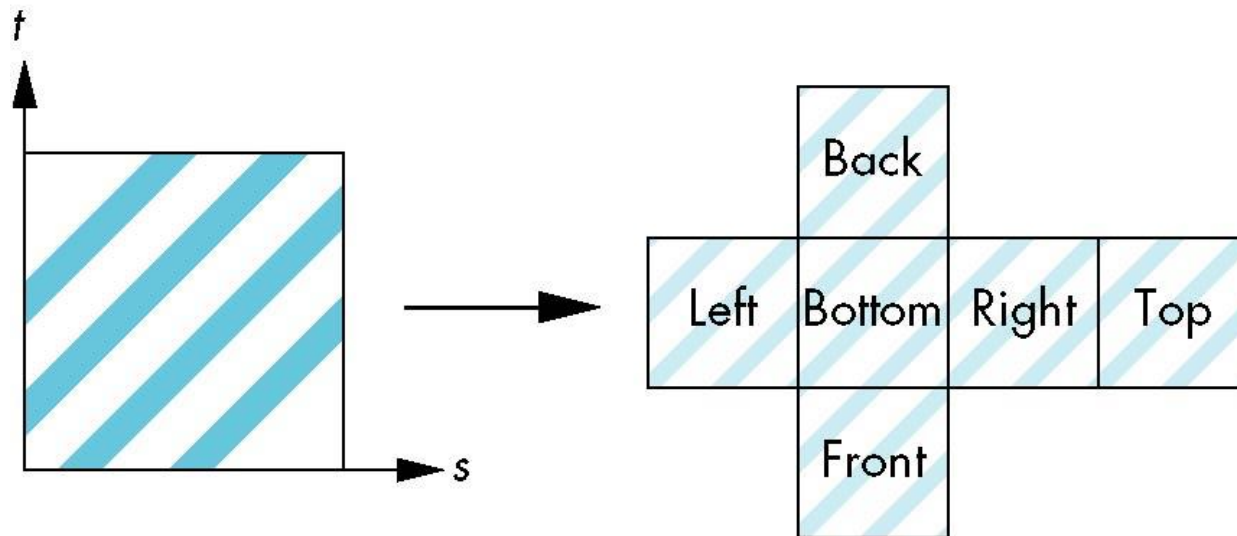
Spherical Projector



courtesy of R. Wolfe

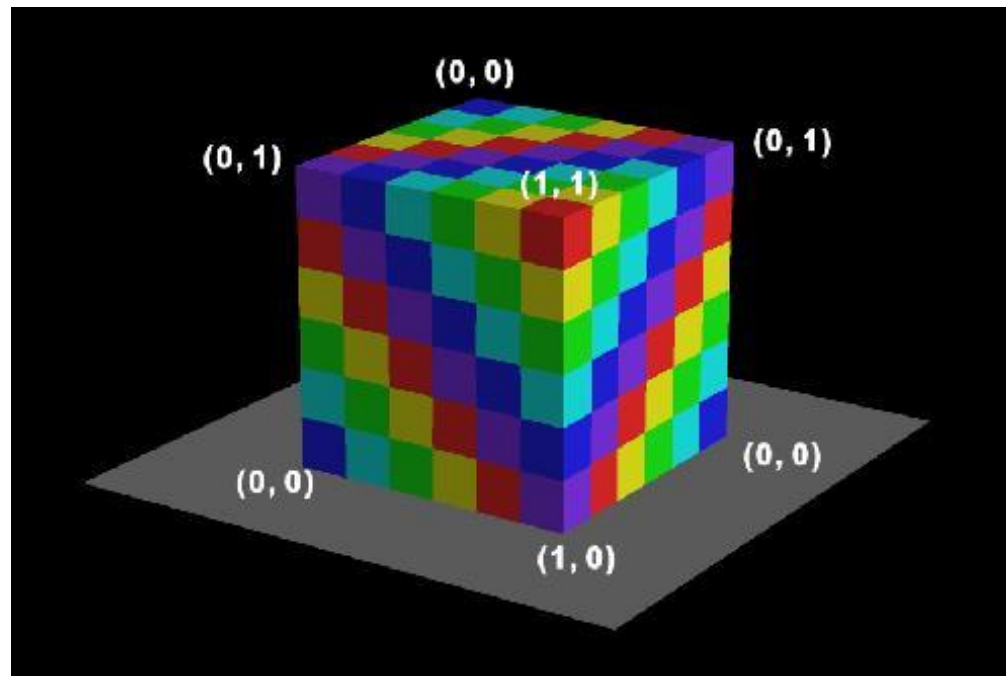
Box Mapping

- Easy to use with simple orthographic projection
- Also used in environment maps



Surface Patches

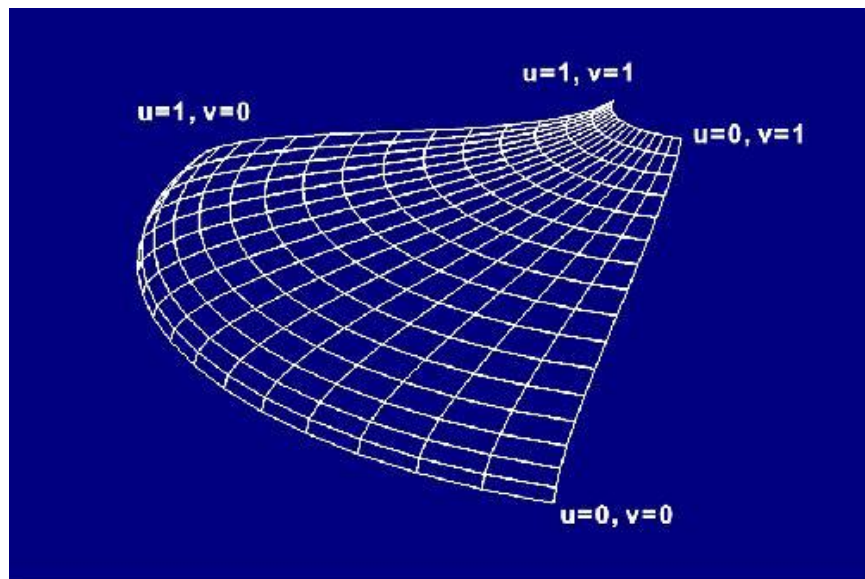
- A polygon or mesh of polygons defining a surface
 - Map four corners of a quad to (u, v) values



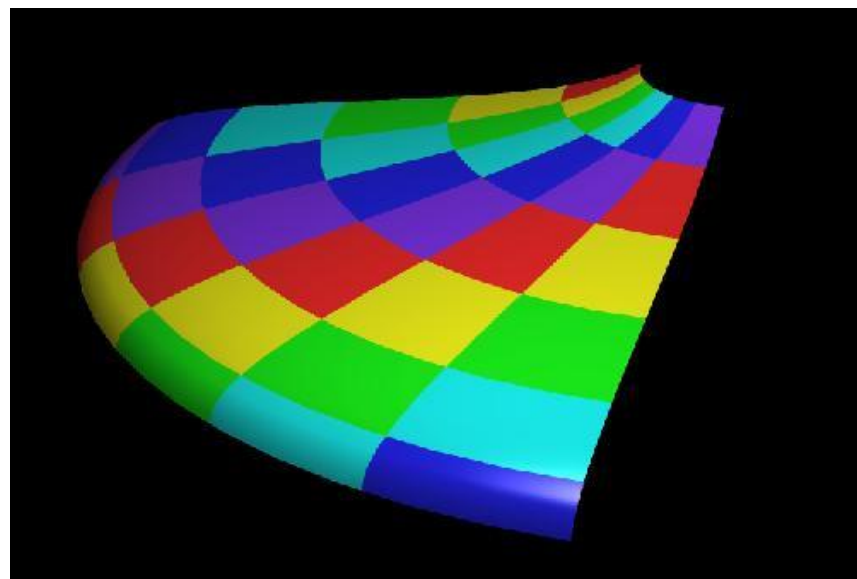
courtesy of
R. Wolfe

Parametric Surfaces

- A parameterized surface patch
 - $x = f(u, v)$, $y = g(u, v)$, $z = h(u, v)$

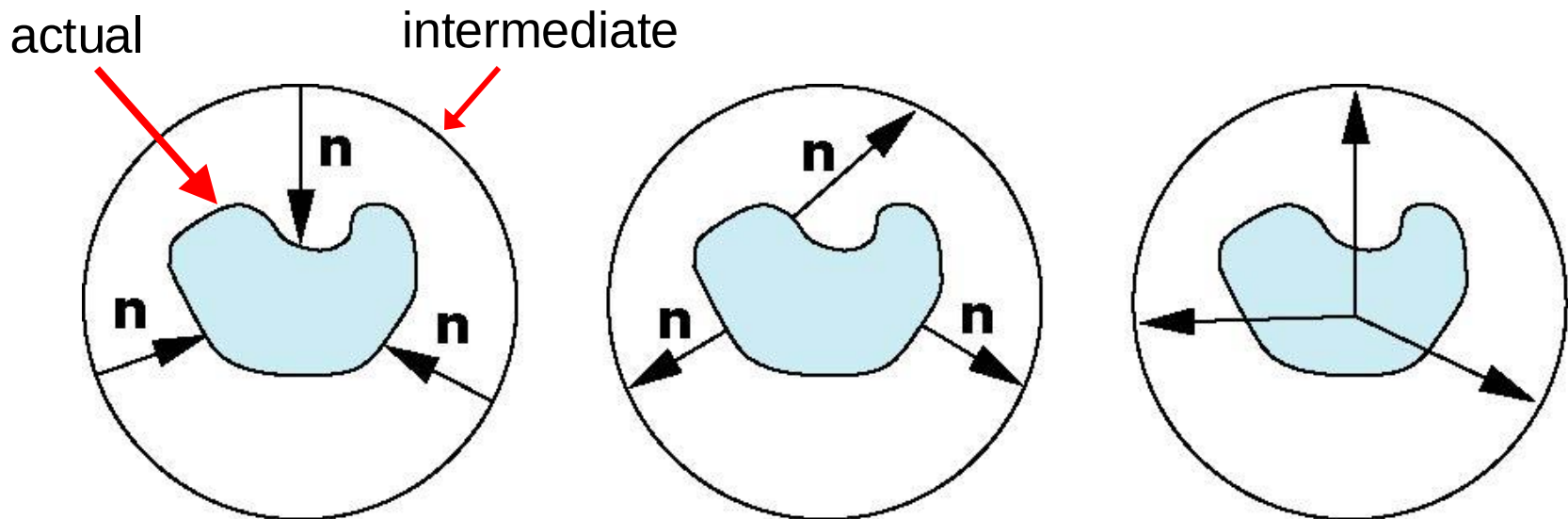


courtesy of R. Wolfe



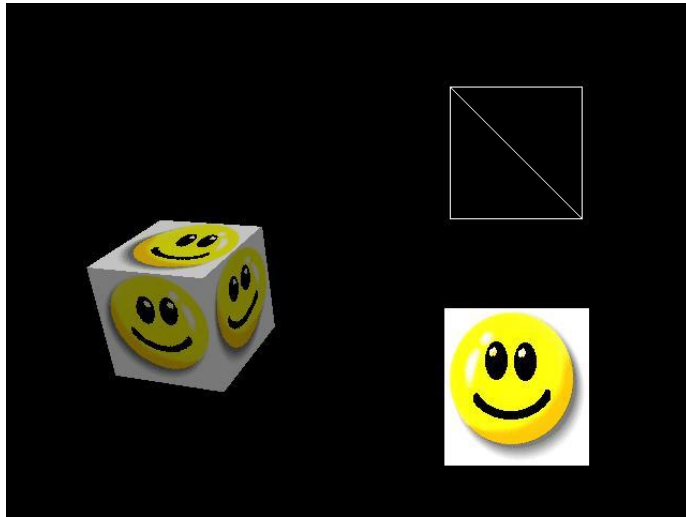
Projectors

- Map between intermediary object and the actual object by using
 1. Normals from intermediate to actual
 2. Normals from actual to intermediate
 3. Vectors from center of intermediate

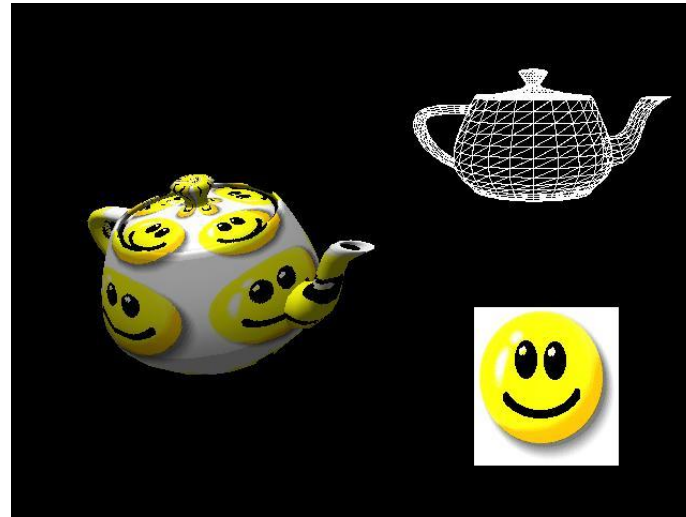


Examples

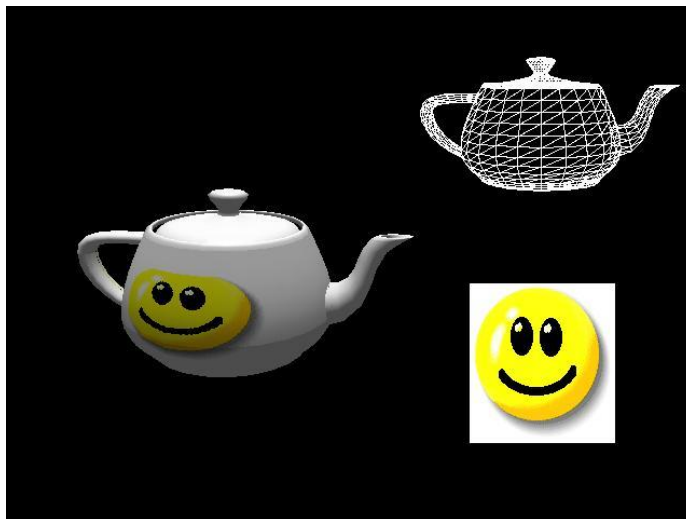
courtesy of Jason Bryan



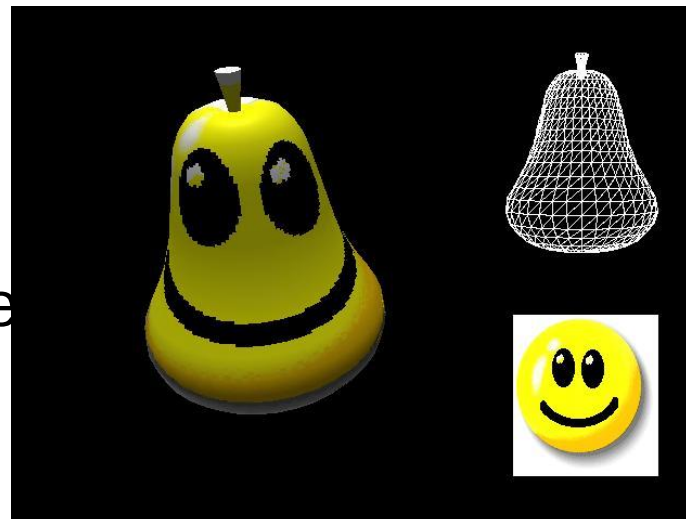
planar



spherical



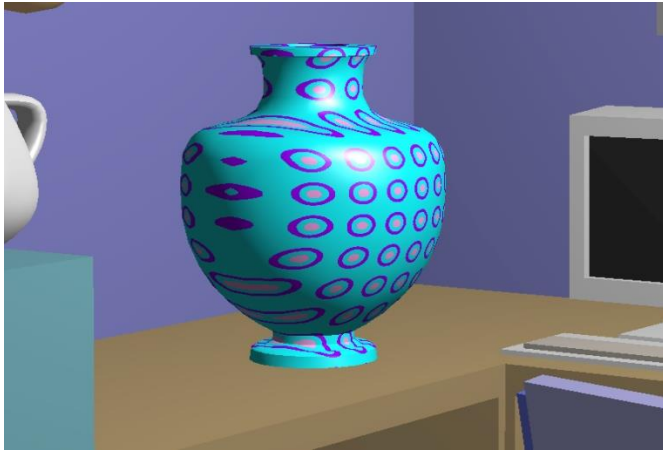
surface
patch



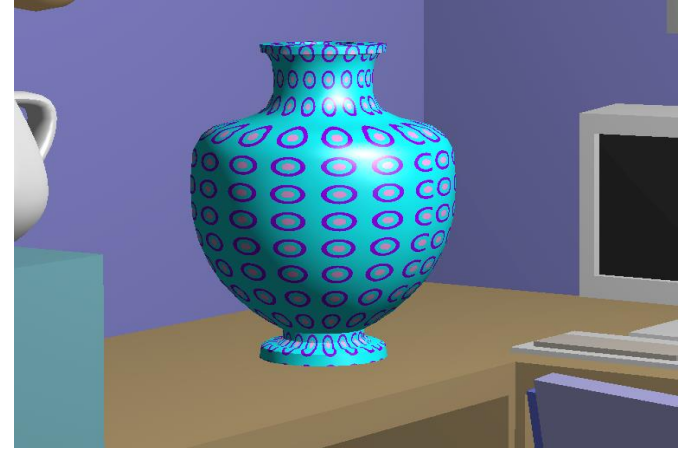
cylindrical

Notice Distortions Due To Object Shape

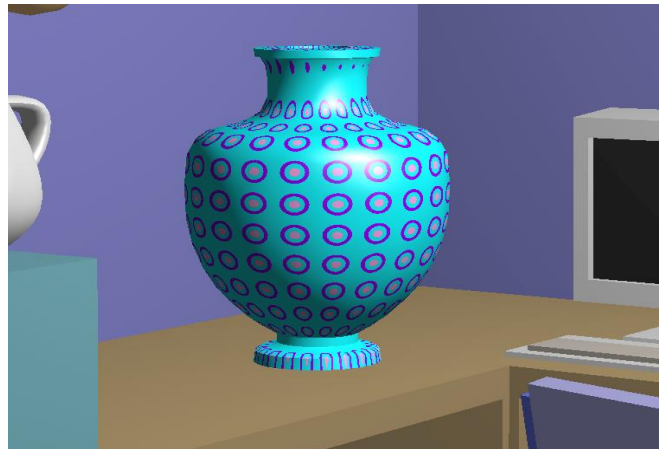
Watt



planar



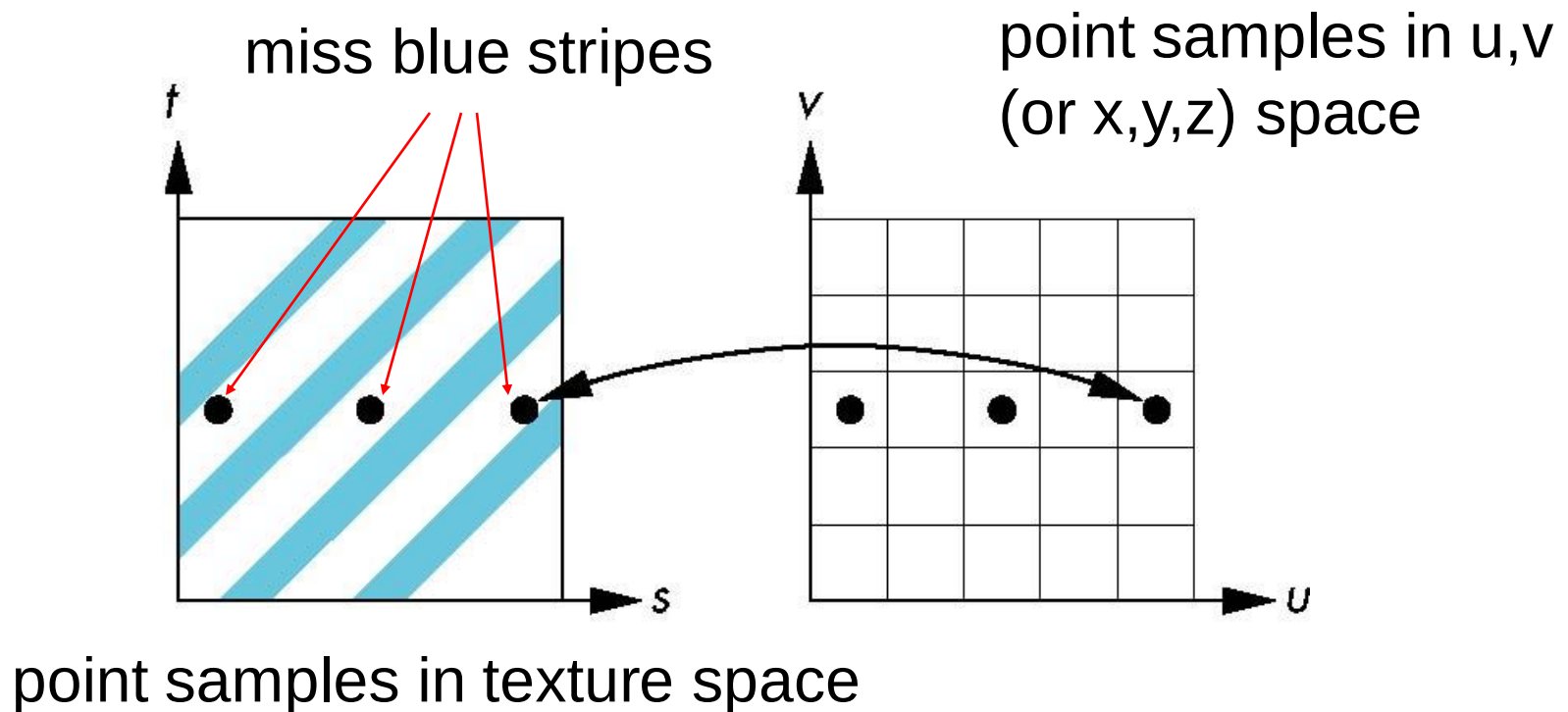
cylindrical



spherical

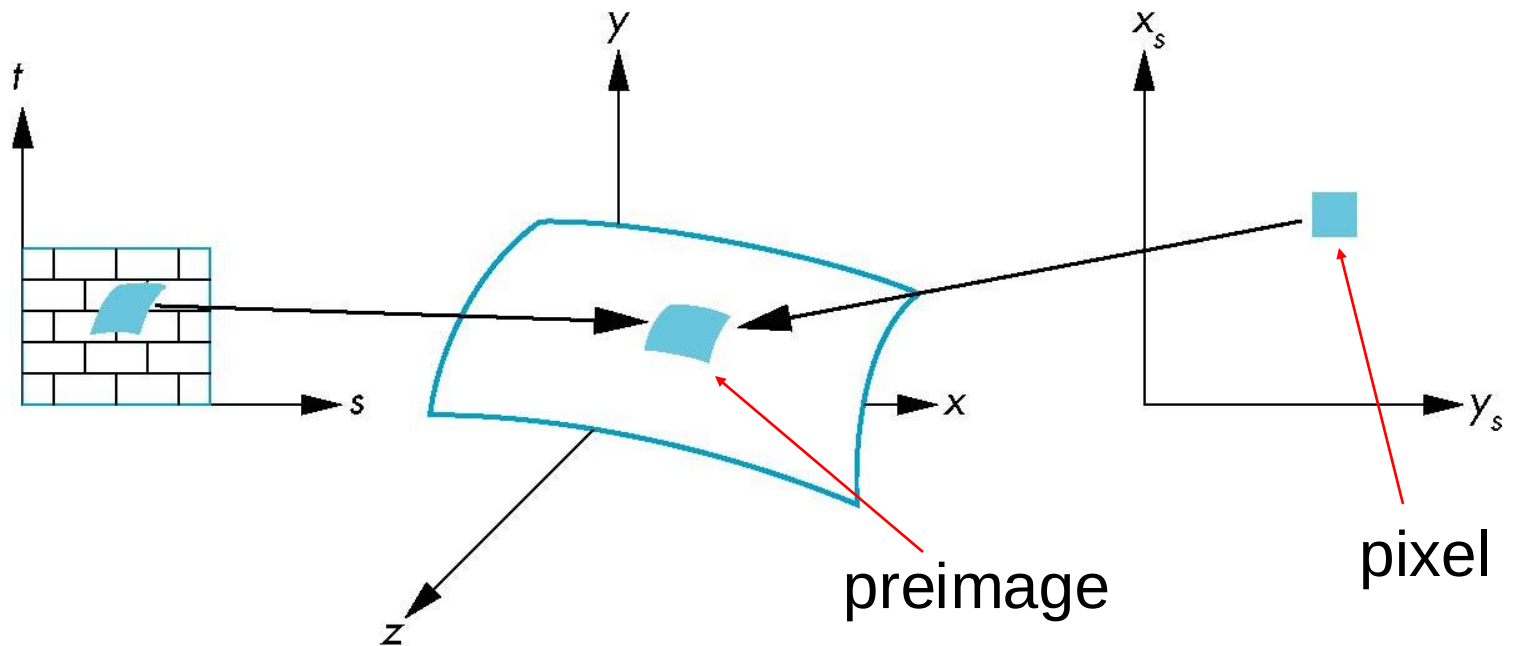
Aliasing

- Point sampling of the texture can lead to aliasing errors



Area Averaging

A better but slower option is to use *area averaging*



Note that *preimage* of pixel is curved

WEBGL TEXTURE MAPPING

Lecturer: Dr. Ufuk Çelikcan

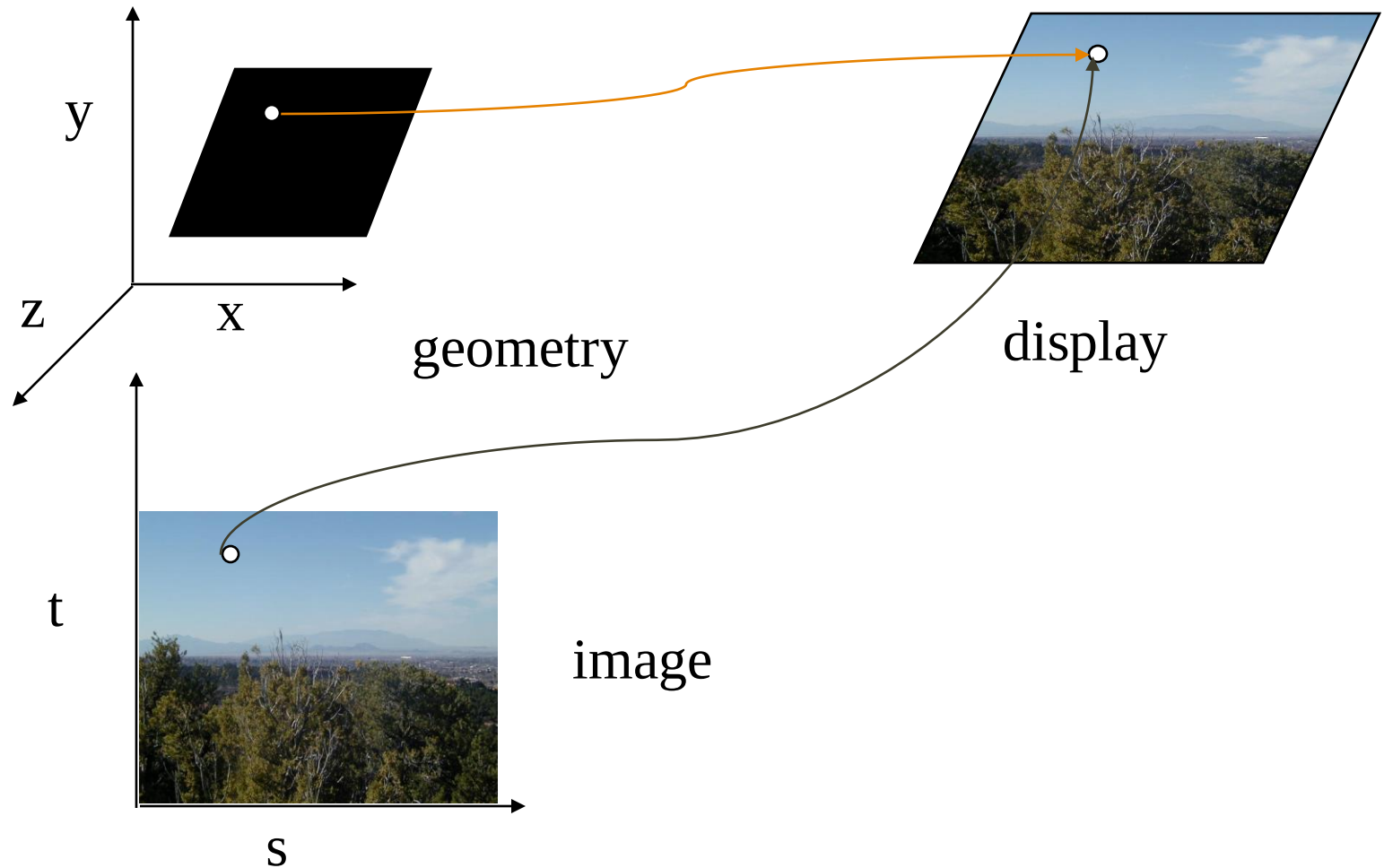
Based on the slides by: E. Angel and D. Shreiner

Basic Strategy

Three steps to applying a texture

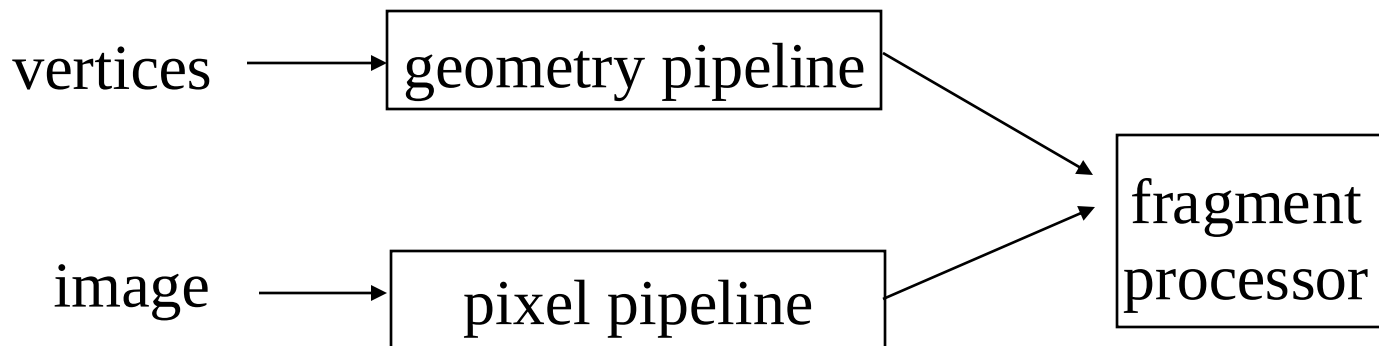
1. specify the texture
 - obtain/generate image
 - assign image to texture
 - enable texturing
2. assign texture coordinates to vertices
 - find/use proper mapping functions
3. specify texture parameters
 - wrapping, filtering ..

Texture Mapping



Texture Mapping and the WebGL Pipeline

- Images and geometry flow through separate pipelines that unite during fragment processing
 - this way, “complex” textures do not affect geometric complexity



Specifying a Texture Image

- Define a texture image from an array of *texels* (texture elements) in CPU memory

```
var my_texels = [];
```

- Define as any other pixel map

- Scanned image
- Generate by application code

- Enable texture mapping

```
gl.enable(GL_TEXTURE_2D)
```

- WebGL supports 1-4 dimensional texture maps

Define Image as a Texture

```
gl.texImage2D( target, level, components,  
              w, h, border, format, type, texels );
```

- Specifying the texels for a texture is done using the `gl.texImage{123}D()` call.
- This will transfer the texels in CPU memory to WebGL, where they will be processed and converted into an internal format.

Define Image as a Texture

```
gl.texImage2D( target, level, components,  
              w, h, border, format, type, texels );
```

target: type of texture, e.g. `gl.TEXTURE_2D`

level: defines how to use the texture image when mapping texels to pixels. Generally, it is set to 0, unless you are using a texturing technique called mipmapping, which we will discuss later.

components: elements per texel, for rgb: 3

w, h: width and height of `texels` in pixels

border: used for smoothing (usually discarded : 0)

format and type: describe texels

texels: pointer to the texel array

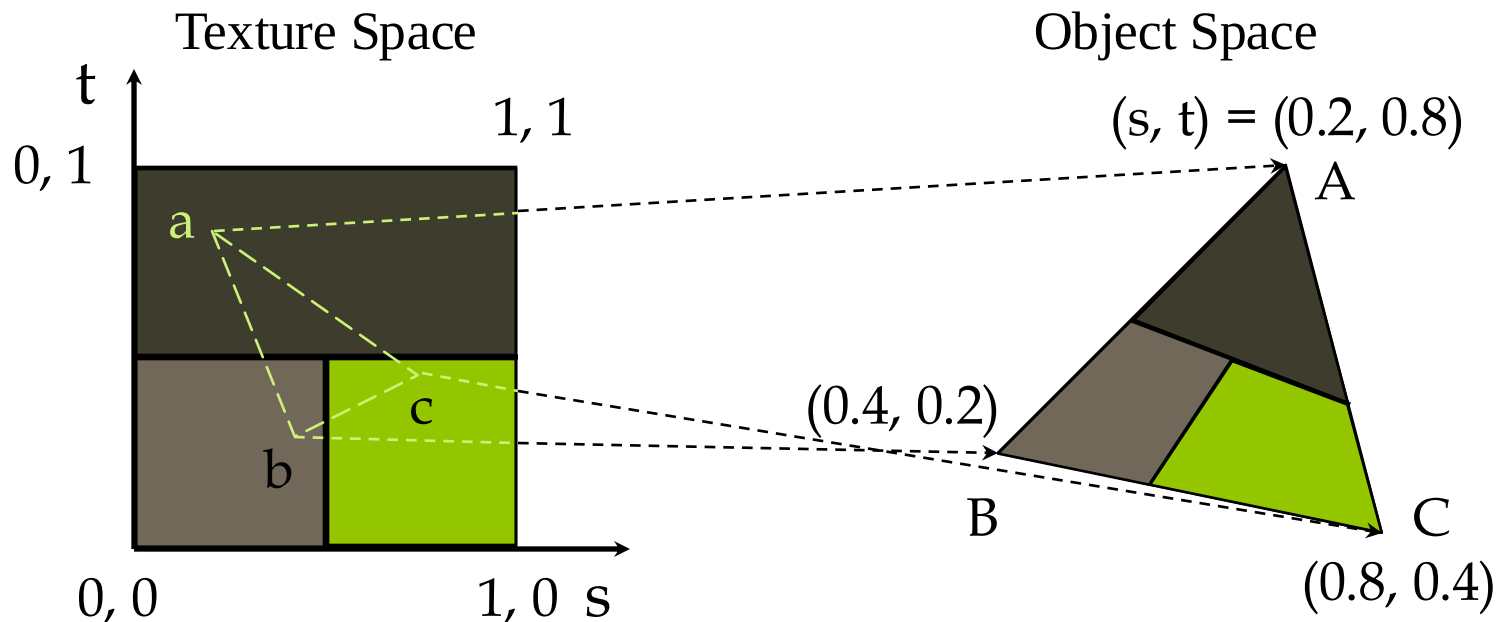
```
gl.texImage2D(gl.TEXTURE_2D, 0, 3, 512, 512, 0, gl.RGBA,  
              gl.UNSIGNED_BYTE, my_texels);
```

Mapping a Texture

- When you want to map a texture onto a geometric primitive, you need to provide texture coordinates.
- Valid texture coordinates are between 0 and 1, for each texture dimension, and usually manifest themselves in shaders as vertex attributes.
 - we will see how to deal with texture coordinates outside the range $[0, 1]$ in a moment.

Mapping a Texture

- Based on parametric texture coordinates
- coordinates need to be specified at each vertex



Typical Code

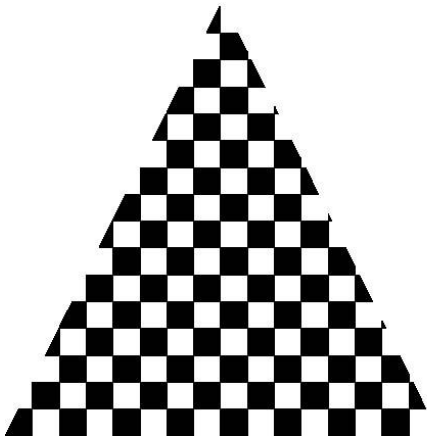
```
offset = 0;
var vPosition = gl.getAttributeLocation( program,
    "vPosition" );
gl.enableVertexAttribArray( vPosition );
gl.vertexAttribPointer( vPosition, 4, gl.FLOAT,
    gl.FALSE, 0, offset );
```

```
offset += vertexCount * 4;
var vTexCoord = gl.getAttributeLocation( program,
    "vTexCoord" );
gl.enableVertexAttribArray( vTexCoord );
gl.vertexAttribPointer( vTexCoord, 2, gl.FLOAT,
    gl.FALSE, 0, offset );
```

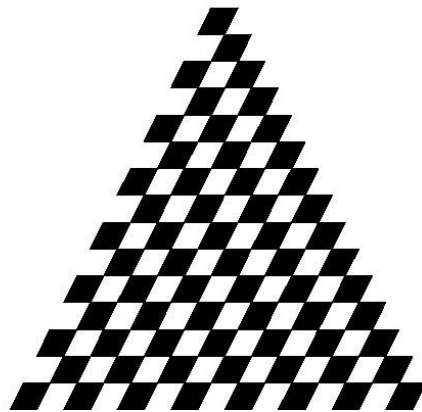
Interpolation

- WebGL uses interpolation to find proper texels from specified texture coordinates
- Need to be careful. Can be distortions

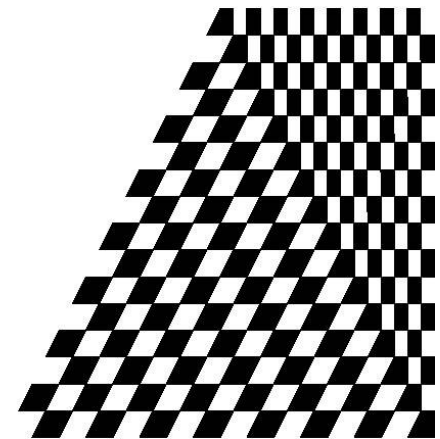
good selection
of tex coordinates



poor selection
of tex coordinates



texture stretched
over trapezoid
showing effects of
bilinear interpolation



Texture Parameters

- OpenGL has a variety of parameters that determine how a texture is applied
 - **Wrapping parameters** determine what happens if texture coordinates s and t are outside the $[0,1]$ range
 - **Filter modes** allow us to use area averaging instead of point samples
 - **Mipmapping** allows us to use textures at multiple resolutions

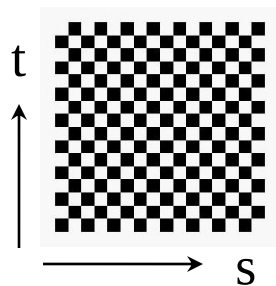
Wrapping Mode

- Clamping: if $s, t > 1$, set s, t to 1

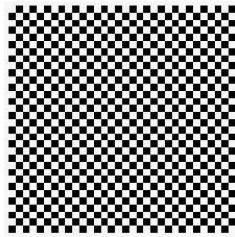
```
gl.texParameteri( gl.TEXTURE_2D,  
                  gl.TEXTURE_WRAP_S, gl.CLAMP )
```

- Repeat: use s, t with modulo 1

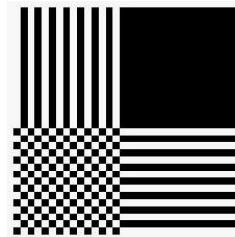
```
gl.texParameteri( gl.TEXTURE_2D,  
                  gl.TEXTURE_WRAP_T, gl.REPEAT )
```



texture



gl.REPEAT
wrapping



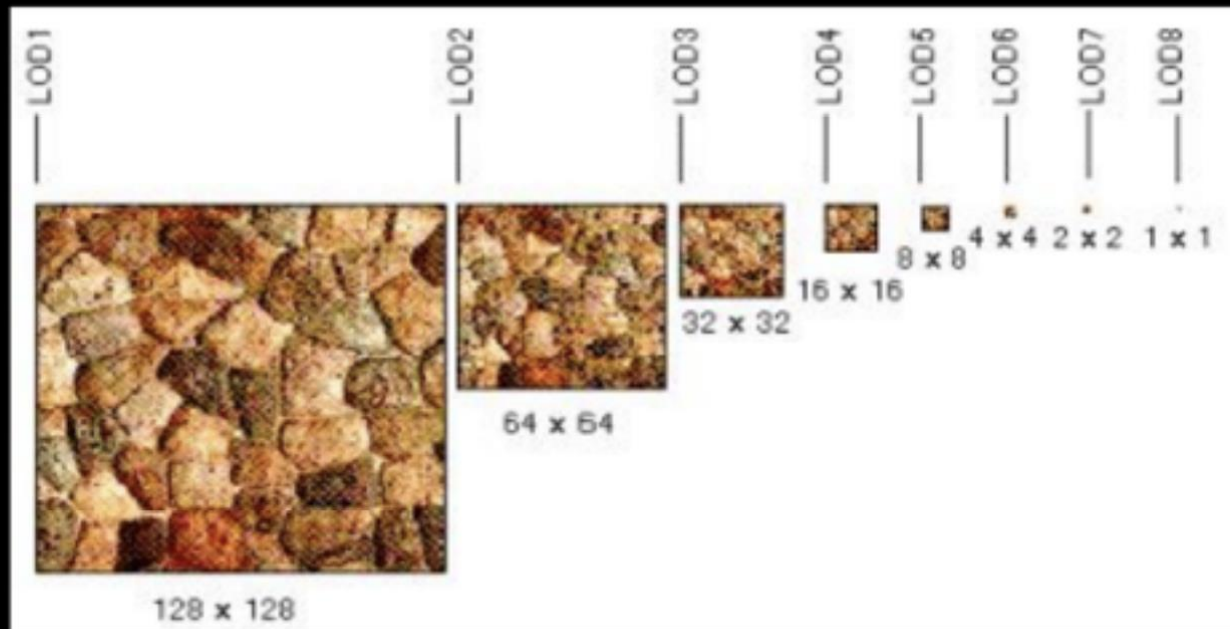
gl.CLAMP
wrapping

Mipmapped Textures

- *Mipmapping* allows for prefiltered texture maps of decreasing resolutions
- Lessens interpolation errors for smaller textured objects

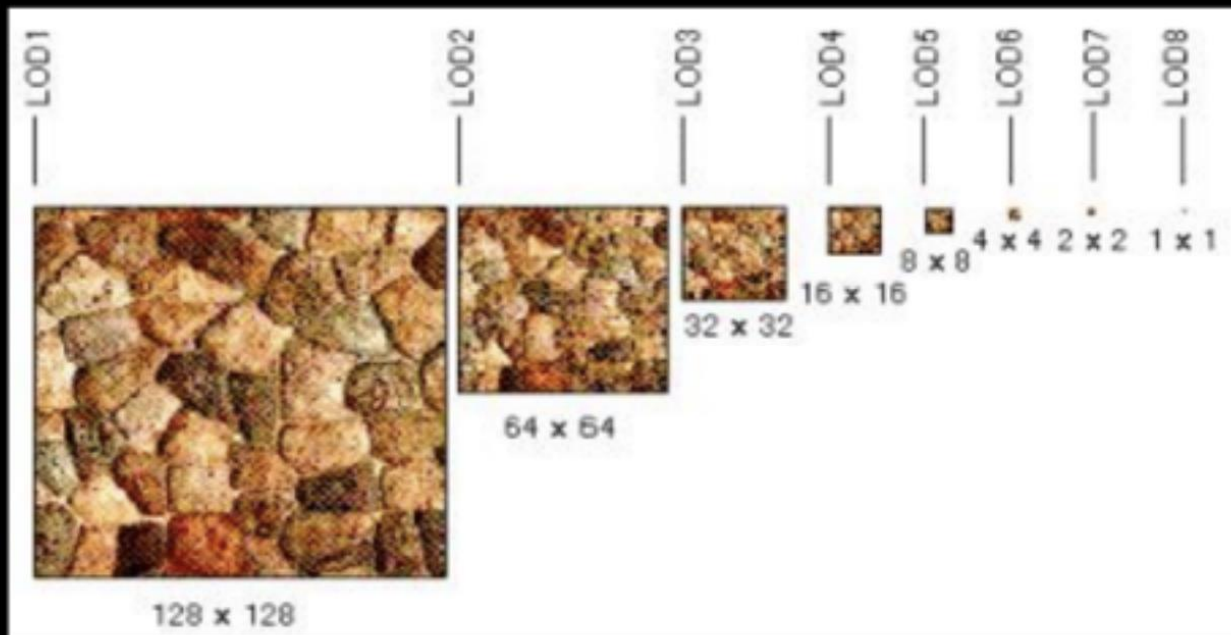
Mipmapping

- Pre-calculate how the texture should look at various distances, then use the appropriate texture at each distance
- Reduces / fixes the aliasing problem



Mipmapping

- Each mipmap (each image below) represents a level of depth (LOD).
- Powers of 2 make things much easier.



The easiest way to using **mipmapping** in your **WebGL** application is to let **WebGL** generate the mipmap chain for you. You only have to load the base texture as level zero, and then you can call the function `gl.generateMipmap()` and **WebGL** automatically generates the complete mipmap chain for you. This is what the code typically looks like for a 2D texture:

```
gl.texImage2D(gl.TEXTURE_2D, 0, gl.RGBA, gl.RGBA, gl.UNSIGNED_BYTE,
              image);
gl.generateMipmap(gl.TEXTURE_2D);
```

The other alternative is to generate the images for the mipmap chain offline with an image editor or a texture tool that is specifically for generating images for **mipmapping**. Then, when you have all the images for your mipmap chain generated, you load each image with `gl.texImage2D()` and specify the mipmap level that the loaded image corresponds to as the second argument to the method. The code to load mipmap levels zero to four could typically look something like this:

```
gl.texImage2D(gl.TEXTURE_2D, 0, gl.RGBA, gl.RGBA, gl.UNSIGNED_BYTE,
              imageLevel_0);

gl.texImage2D(gl.TEXTURE_2D, 1, gl.RGBA, gl.RGBA, gl.UNSIGNED_BYTE,
              imageLevel_1);

gl.texImage2D(gl.TEXTURE_2D, 2, gl.RGBA, gl.RGBA, gl.UNSIGNED_BYTE,
              imageLevel_2);

gl.texImage2D(gl.TEXTURE_2D, 3, gl.RGBA, gl.RGBA, gl.UNSIGNED_BYTE,
              imageLevel_3);

gl.texImage2D(gl.TEXTURE_2D, 4, gl.RGBA, gl.RGBA, gl.UNSIGNED_BYTE,
              imageLevel_4);
```

As you can see, this second approach requires some more work. However, it is good to know that this option exists if you have very specific requirements on how the lower-resolution images in the mipmap chain are generated.

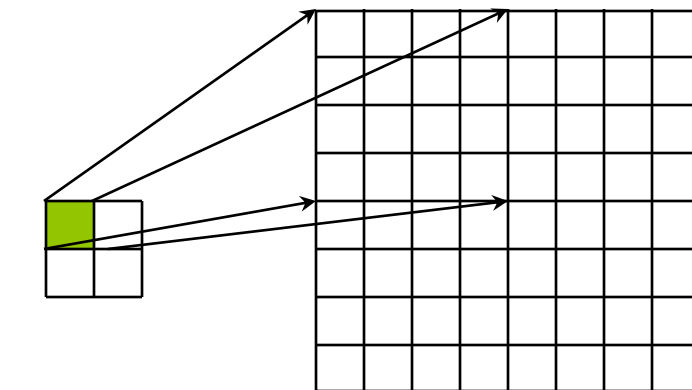


*To use **mipmapping**, both dimensions of the base texture must be a power of two (1, 2, 4, 8, 16, ..., 512, 1024, and so on).*

Magnification and Minification

- A single texel can cover more than one pixel : ***magnification*** or
- More than one texel can cover a single pixel : ***minification***

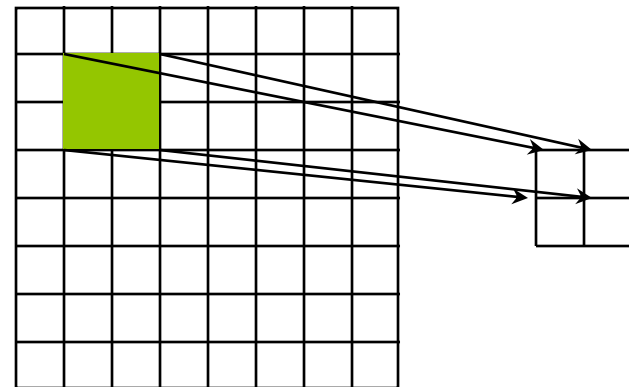
Can use point sampling (nearest texel) or linear filtering (2 x 2 filter) to obtain texture values



Texture

Polygon

Magnification



Texture

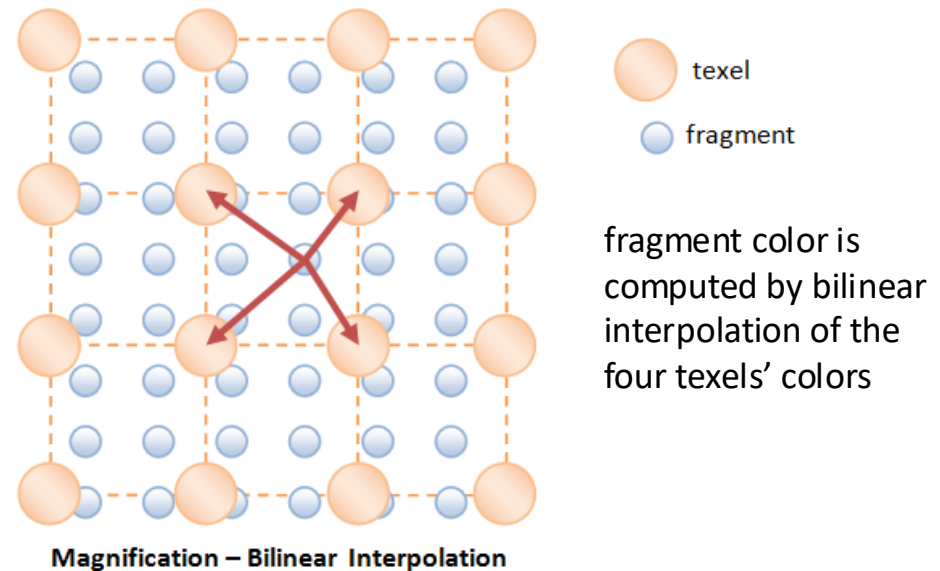
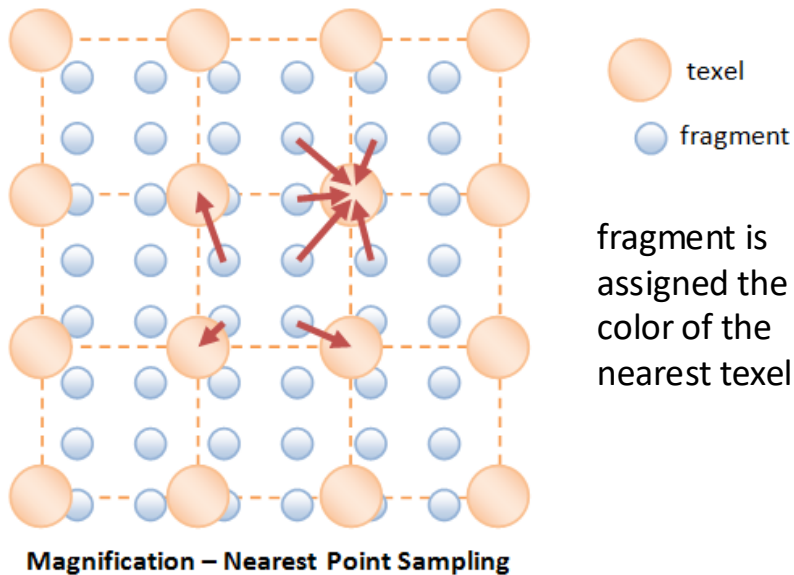
Polygon

Minification

Magnification and Minification

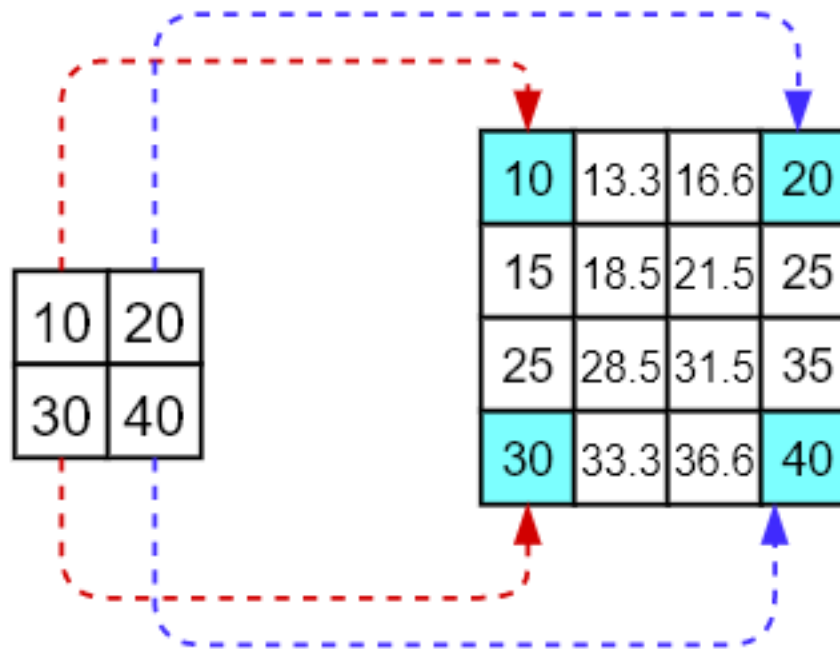
- A single texel can cover more than one pixel : ***magnification*** or
- More than one texel can cover a single pixel : ***minification***

Can use point sampling (nearest texel) or linear filtering (2 x 2 filter) to obtain texture values



Bilinear Interpolation

- 2x2 image patch is mapped to a 4x4 region using bilinear interpolation



Filter Modes

determined by

- `gl.texParameteri( target, type, mode )`

```
gl.texParameteri( gl.TEXTURE_2D, gl.TEXTURE_MAG_FILTER,  
                  gl.NEAREST ) ;
```

```
gl.texParameteri( gl.TEXTURE_2D, gl.TEXTURE_MIN_FILTER,  
                  gl.LINEAR ) ;
```

Note that linear filtering requires a border of an extra texel for filtering at edges >> `border = 1`

Filter Modes

GL_TEXTURE_MIN_FILTER accepts the following options:

- GL_NEAREST corresponds to nearest-neighbour rendering
- GL_LINEAR corresponds to bilinear filtering
- GL_NEAREST_MIPMAP_NEAREST
- GL_NEAREST_MIPMAP_LINEAR
- GL_LINEAR_MIPMAP_NEAREST
- GL_LINEAR_MIPMAP_LINEAR

GL_TEXTURE_MAG_FILTER accepts the following options:

- GL_NEAREST
- GL_LINEAR

Filter Modes

When you have a complete mipmap chain for your texture, you can select four new filters for the minification in addition to the non-mipmapped filters `gl.NEAREST` and `gl.LINEAR`.

- `gl.NEAREST` — Nearest neighbor filtering is used on the base texture. This means that the color for the texel nearest to the texture coordinate is used.
- `gl.LINEAR` — Linear filtering (also called bilinear filtering) is used on the base texture. This means that the color is calculated as the weighted average of the four texels surrounding the texture coordinate.
- `gl.NEAREST_MIPMAP_NEAREST` — takes the nearest mipmap to match the pixel size and uses nearest neighbor interpolation for texture sampling.
- `gl.LINEAR_MIPMAP_NEAREST` — takes the nearest mipmap level and samples that level using linear interpolation.
- `gl.NEAREST_MIPMAP_LINEAR` — linearly interpolates between the two mipmaps that most closely match the size of a pixel and samples the interpolated level via nearest neighbor interpolation.
- `gl.LINEAR_MIPMAP_LINEAR` — linearly interpolates between the two closest mipmaps and samples the interpolated level via linear interpolation.

Filter Modes

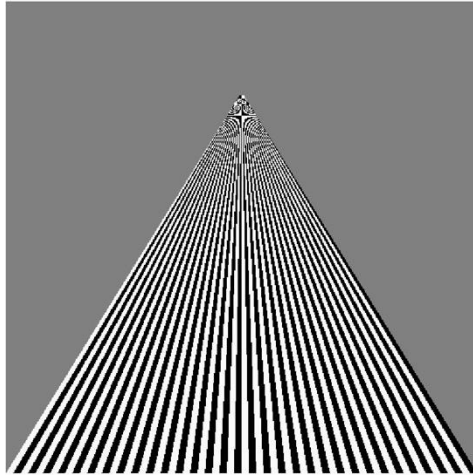
The four filters that use mipmapping all have names of the form `gl.A_MIPMAP_B`, where A and B can be either NEAREST or LINEAR.

- Here A specifies how the texels are used within one mipmap level. If A is NEAREST, it means that the nearest neighbor is used within a mipmap level. If A is LINEAR, it means that linear interpolation is used within a mipmap level.
- B specifies if a single mipmap level is used or if the two closest mipmap levels are used. If B is NEAREST it means that the closest mipmap level is used, and if B is LINEAR it means that the two closest mipmap levels are used and within these levels the method specified by A is used. The result from the two mipmap levels is then linearly interpolated to get the final result.

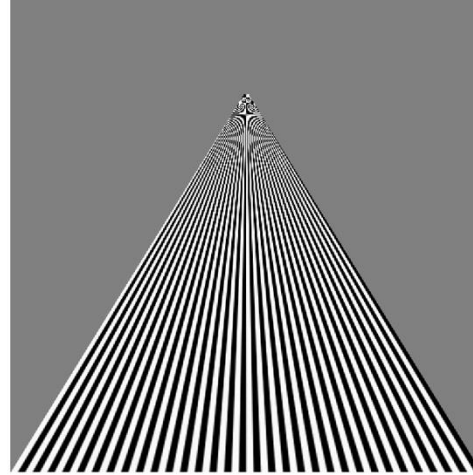
Example

https://www.cs.unm.edu/~angel/BOOK/INTERACTIVE_COMPUTER_GRAPHICS/SIXTH_EDITION/CODE/WebGL/CODE/07/textureSquare.html

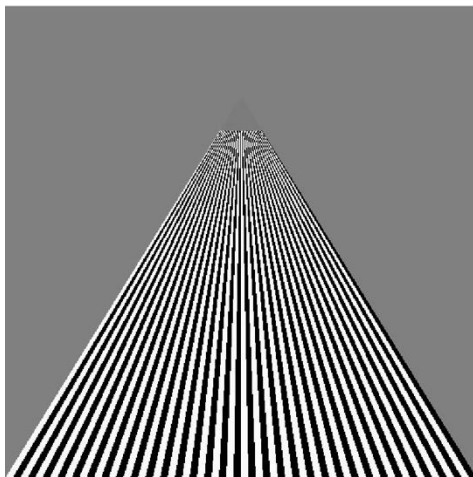
point
sampling



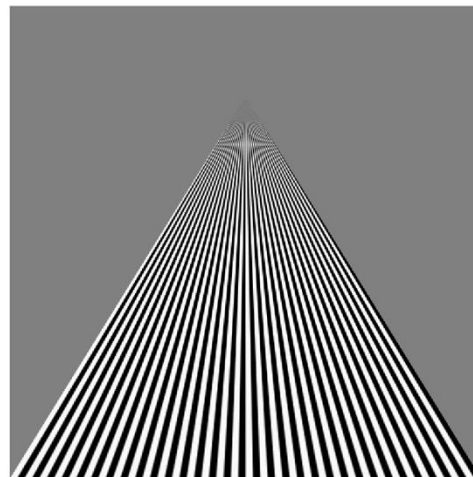
linear
filtering



mipmapped
point
sampling

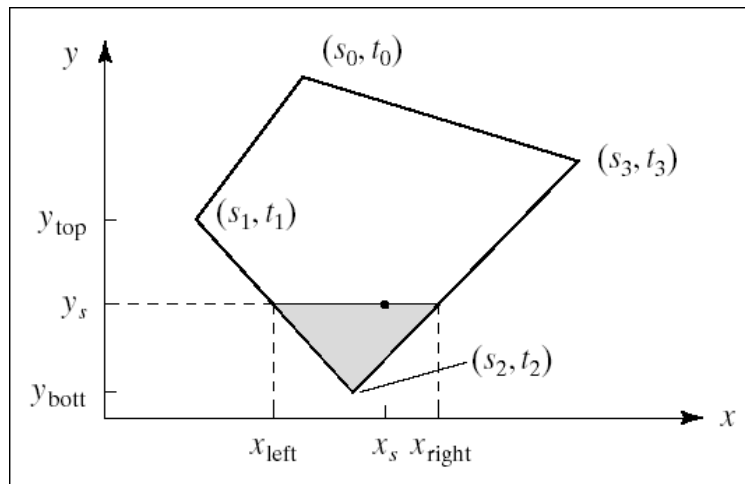


mipmapped
linear
filtering



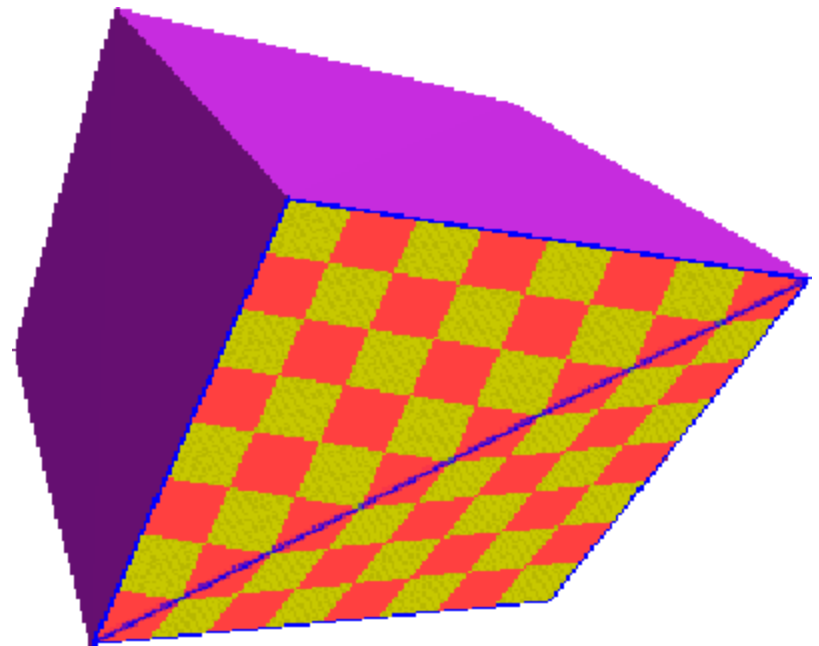
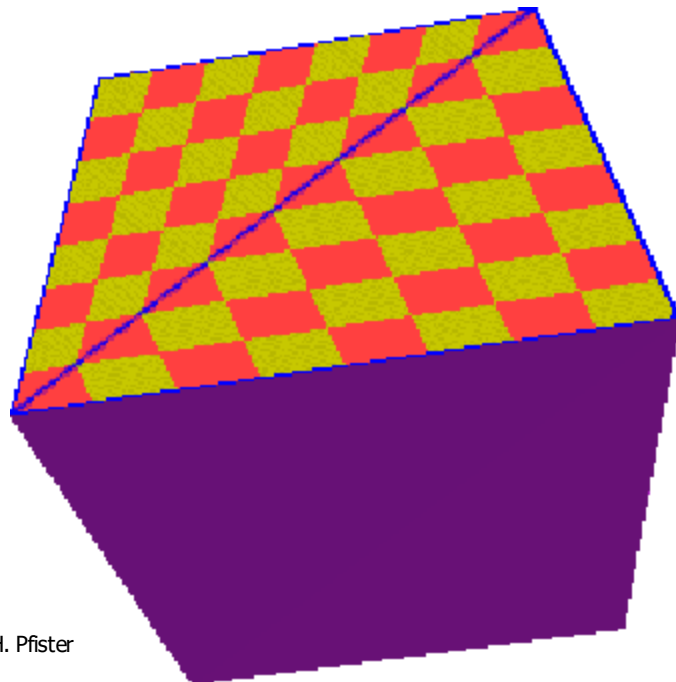
Texture Rasterization

- Texture coordinates are interpolated from polygon vertices just like ... remember ...
 - Color : Gouraud shading
 - Depth: Z-buffer
- First along polygon edges between vertices
- Then along scanlines between left and right sides



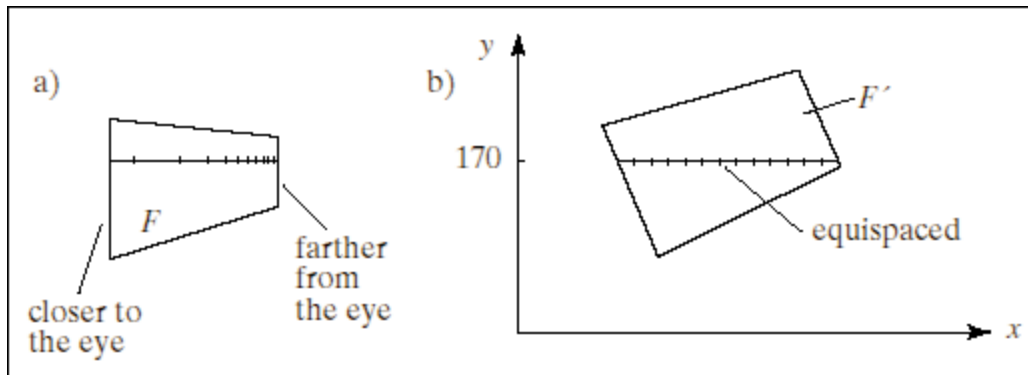
Linear Texture Coordinate Interpolation

- This doesn't work in perspective projection!
- The textures look warped along the diagonal
- Noticeable during an animation



Why?

- Equal spacing in screen (pixel) space is **not** the same as in texture space in perspective projection
 - **Perspective foreshortening**



from Hill

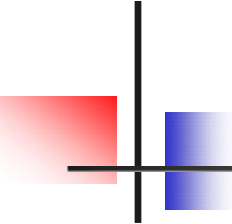


courtesy of
H. Pfister

Perspective-Correct Texture Coordinate Interpolation

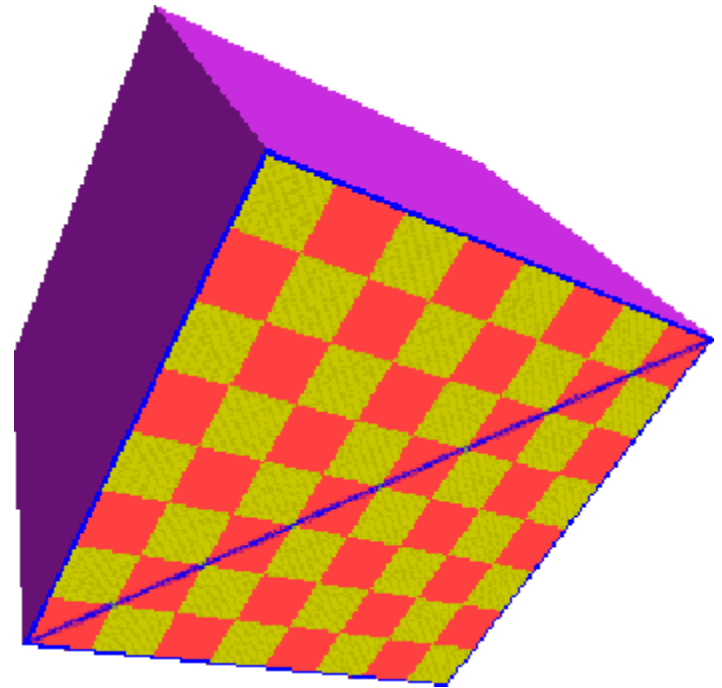
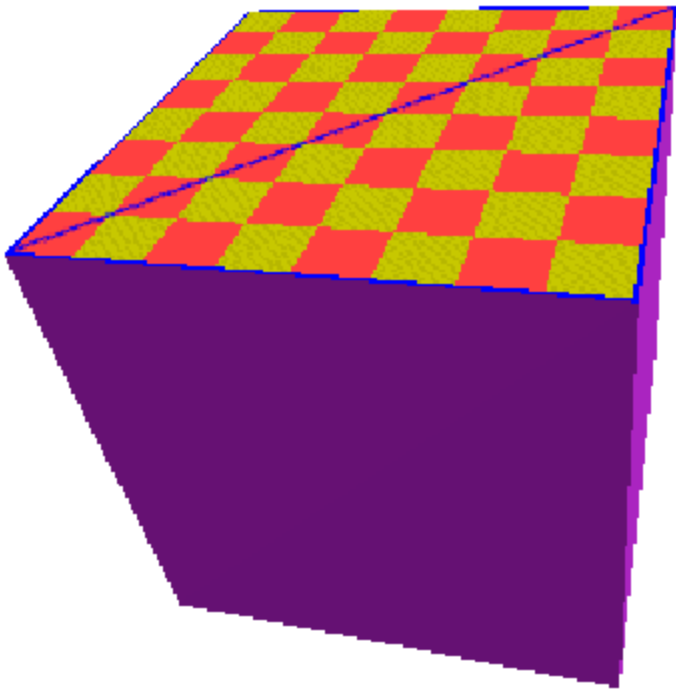
Interpolate $(\text{tex_coord}/w)$ over the polygon, then do perspective divide after interpolation

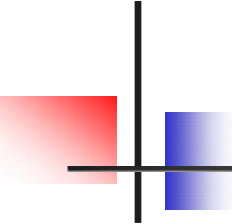
1. Compute at each vertex after perspective transformation
 1. Numerators $s/w, t/w$
 2. Denominator $1/w$
2. Linearly interpolate $1/w, s/w$, and t/w across the polygon
3. At each pixel
 - Perform perspective division of interpolated texture coordinates $(s/w, t/w)$ by interpolated $1/w$ (i.e., numerator over denominator) to get (s, t)



Perspective-Correct Interpolation

- That fixed it!





Perspective Correction Hint

- Texture coordinate and color interpolation:
 - Linearly in screen space (wrong) **OR**
 - Perspective correct interpolation (slower)
- **glHint** (GL_PERSPECTIVE_CORRECTION_HINT, **hint**), where **hint** is one of:
 - GL_NICEST: Perspective
 - GL_FASTEST: Linear
 - GL_DONT_CARE: Linear

Texture Objects

- WebGL has *texture objects*
 - one image per texture object
 - Texture memory can hold multiple texture objects

Applying Textures

1. specify textures in texture objects
2. set texture filter
3. set texture function
4. set texture wrap mode
5. optional: set perspective correction hint
6. bind texture object
7. enable texturing
8. supply texture coordinates for vertex

Texture Objects

- Have WebGL store your images
 - one image per texture object
 - may be shared by several graphics contexts
- The first step in creating texture objects is to have WebGL reserve some indices for your objects

```
gl.createTextures();
```

Texture Objects (cont'd.)

- Bind textures before using.
- To have WebGL use a particular texture object, call `gl.bindTexture()` with the target and id of the object you want to make active.

```
gl.bindTexture( target, id );
```

- The target is one of `gl.TEXTURE_2D()`
- To delete texture objects, use

```
gl.deleteTexture( id );
```

where `texIds` is an array of texture object identifiers to be deleted.

Applying a Texture in a Shader

- Just like vertex attributes are associated with data in the application, so with textures.
- In particular, you access a texture defined in your application using a ***texture sampler*** in your shader.
- The type of the sampler needs to match the type of the associated texture.
- For example, you would use a sampler2D to work with a two-dimensional texture created with `gl.texImage2D( gl.TEXTURE_2D, ... );`

```
attribute vec4 texCoord;
```

```
// Declare the sampler
```

```
uniform float      intensity;
```

```
uniform sampler2D diffuseMaterialTexture;
```

```
// Apply the material color
```

```
vec3 diffuseColor = intensity *  
    texture2D(diffuseMaterialTexture, texCoord).rgb;
```

Applying a Texture in a Shader

- Within the shader, you use the ***texture2D()*** function to retrieve data values from the texture associated with your sampler.
- To the texture2D() function, you pass the sampler as well as the texture coordinates where you want to pull the data from.

```
attribute vec4 texCoord;
```

```
// Declare the sampler
```

```
uniform float      intensity;
```

```
uniform sampler2D diffuseMaterialTexture;
```

```
// Apply the material color
```

```
vec3 diffuseColor = intensity *
```

```
    texture2D(diffuseMaterialTexture, texCoord).rgb;
```

**Now let's see our cube example
with texturing**

Applying Texture to Cube

- Need to provide texture coordinates for use in our shaders.
- Following our previous example, we merely add an additional vertex attribute that contains our texture coordinates. We do this for each vertex.
- We will also need to update VBOs and shaders to take this new attribute into account.

```
// add texture coordinate attribute to  
quad function
```

```
quad( int a, int b, int c, int d )  
{  
    vColors[Index] = colors[a];  
    vPositions[Index] = positions[a];  
    vTexCoords[Index] = vec2( 0.0, 0.0 );  
    Index++;  
  
    vColors[Index] = colors[b];  
    vPositions[Index] = positions[b];  
    vTexCoords[Index] = vec2( 1.0, 0.0 );  
    Index++;  
    ... // rest of vertices  
}
```

Texture Object

- Below code completely specifies a texture object.
- The code creates a texture id by calling **gl.createTexture()**
- It then binds the texture using **gl.bindTexture()** to enable the object for use, and loading in the texture by calling **gl.texImage2D()**
- After that, sampler characteristics are set

```
var texture = gl.createTexture(); //create a texture object
```

```
gl.bindTexture( gl.TEXTURE_2D, texture ); //bind the texture object as the target  
gl.pixelStorei(gl.UNPACK_FLIP_Y_WEBGL, true); //flip the image's y-axis
```

```
gl.texImage2D( gl.TEXTURE_2D, 0, gl.RGBA, TextureSize, //set the texture image  
              TextureSize, gl.RGBA, gl.UNSIGNED_BYTE, image );
```

```
gl.generateMipmap( gl.TEXTURE_2D ); //generate the mipmaps  
gl.texParameteri(gl.TEXTURE_2D,  
                gl.TEXTURE_MAG_FILTER, gl.NEAREST ); //set the texture parameters  
gl.texParameteri(gl.TEXTURE_2D,  
                gl.TEXTURE_MIN_FILTER, gl.NEAREST_MIPMAP_LINEAR );  
gl.activeTexture( gl.TEXTURE0 ); //enable texture unit 0  
gl.uniform1i(gl.getUniformLocation(shaderProgram, "textureImage"), 0);
```

//set the texture unit 0 to the texture sampler

Typical Code

```
offset = 0;
var vPosition = gl.getAttributeLocation(shaderProgram,
    "vPosition" );
gl.enableVertexAttribArray( vPosition );
gl.vertexAttribPointer( vPosition, 4, gl.FLOAT,
    gl.FALSE, 0, offset);
```

```
offset += points*4;
var vTexCoord = gl.getAttributeLocation(shaderProgram,
    "vTexCoord" );
gl.enableVertexAttribArray( vTexCoord );
gl.vertexAttribPointer( vTexCoord, 2, gl.FLOAT,
    gl.FALSE, 0, offset );
```

Vertex Shader

```
in vec4 vPosition;
in vec4 vColor;
in vec2 vTexCoord;

out vec4 color;
out vec2 texCoord;

void main()
{
    color      = vColor;
    texCoord   = vTexCoord;
    gl_Position = vPosition;
}
```

- In order to apply textures to our geometry, we need to modify both the vertex shader and the fragment shader.
- we add some simple logic to pass-thru the texture coordinates from an attribute into data for the rasterizer.

Fragment Shader

- Continuing to update our shaders, we add some simple code to modify our fragment shader to include sampling a texture.
- How the texture is sampled (e.g., coordinate wrap modes, texel filtering, etc.) is configured in the application using the `glTexParameter*()` call.

```
precision mediump float;
```

```
in vec4 color;
```

```
in vec2 texCoord;
```

```
out vec4 fColor;
```

```
uniform sampler2D textureImage;
```

```
void main()
```

```
{
```

```
    fColor = color * texture2D(textureImage, texCoord);
```

```
}
```

Let's see it in action

https://www.cs.unm.edu/~angel/BOOK/INTERACTIVE_COMPUTER_GRAPHICS/SIXTH_EDITION/CODE/WebGL/CODE/07/textureCube1.html

take a look at the page source of above page for shaders' implementation

SHADER APPLICATIONS

Lecturer: Dr. Ufuk Çelikcan

Based on the slides by: E. Angel and D. Shreiner

Vertex Shader Applications

- Moving vertices
 - Morphing
 - Wave motion
 - Fractals
- Lighting
 - More realistic models
 - Cartoon shaders

Simple **Wave Motion** Vertex Shader

```
uniform float time;
uniform float xs, zs, // frequencies
uniform float h; // height scale
uniform mat4 ModelView, Projection;
in vec4 vPosition;

void main() {
    vec4 t = vPosition;
    t.y = vPosition.y
        + h*sin(time + xs*vPosition.x)
        + h*sin(time + zs*vPosition.z);
    gl_Position = Projection*ModelView*t;
}
```

Simple Particle System Vertex Shader

```
uniform vec3 init_vel;  
uniform float g, t;  
uniform mat4 Projection, ModelView;  
in vec4 vPosition;
```

for free fall:
 $\Delta y = v_i t - gt^2/2$

```
void main()  
{
```

```
    vec3 object_pos;
```

```
    object_pos.x = vPosition.x + init_vel.x*t;
```

```
    object_pos.y = vPosition.y + init_vel.y*t - g*t*t / 2.0;
```

```
    object_pos.z = vPosition.z + init_vel.z*t;
```

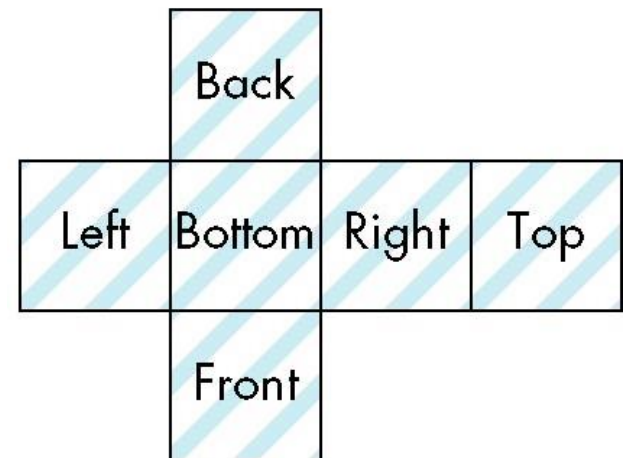
```
    gl_Position = Projection*ModelView*vec4(object_pos, 1);
```

```
}
```

The only force on a particle
in air is the gravitational force
which is in $-y$ direction

Cube Maps

- We can form a cube map texture by defining six 2D texture maps that correspond to the sides of a box
- Supported by WebGL
- Also supported in GLSL through cubemap sampler
 - `vec4 texColor = textureCube(mycube, texcoord);`
 - `textureCube` >> **texture** in new GLSL versions
- Texture coordinates must be 3D

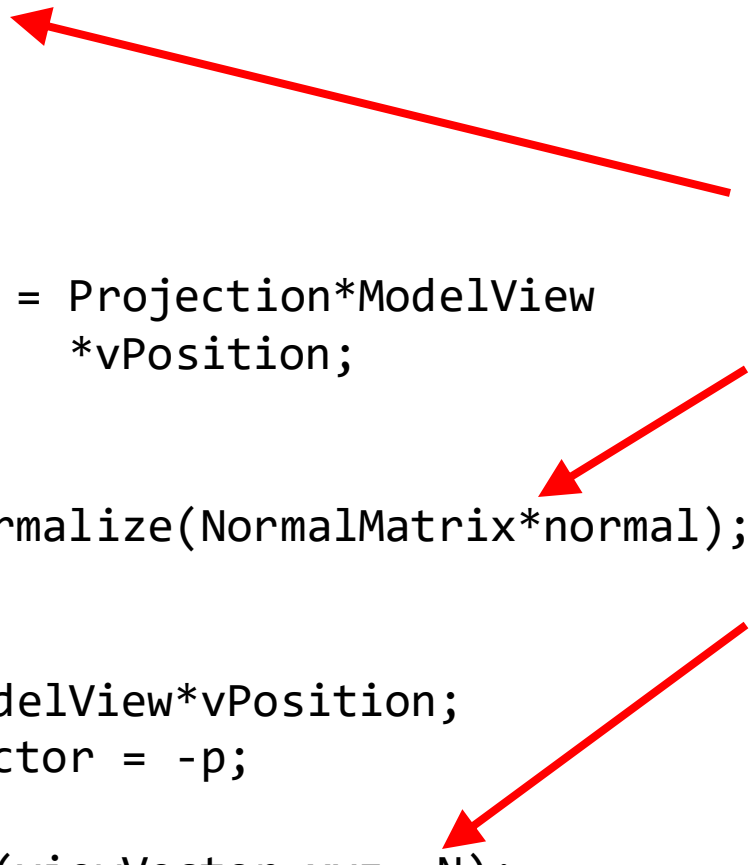


Environment Maps with Shaders

- Environment maps are usually computed in world coordinates which can differ from object coordinates because of the modeling matrix
 - May have to keep track of modeling matrix and pass it to the shader as a uniform variable
- Can also use reflection map or refraction map (for example: to simulate water)

Reflection Map Vertex Shader

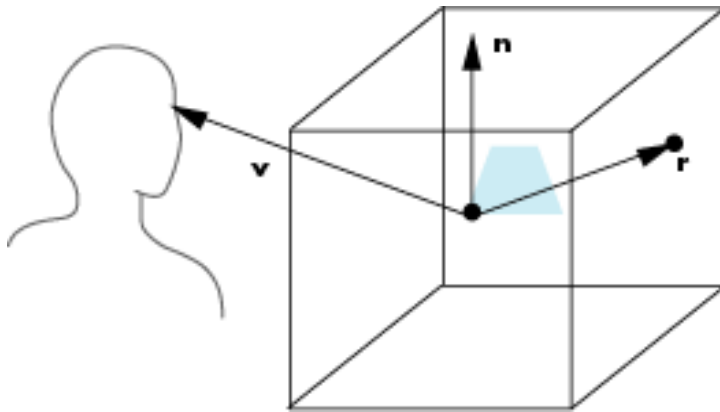
```
uniform mat4 Projection, ModelView,  
NormalMatrix;  
in vec4 vPosition;  
in vec4 normal;  
out vec3 R;  
  
void main(void)  
{  
    gl_Position = Projection*ModelView  
                  *vPosition;  
  
    vec3 N = normalize(NormalMatrix*normal);  
  
    vec4 p = ModelView*vPosition;  
    vec4 viewVector = -p;  
  
    R = reflect(viewVector.xyz, N);  
}
```



- We can compute the reflection vector at each vertex in our vertex shader and then let the rasterizer interpolate these values over the given primitive.
- However, to compute the reflection vector, we need the normal to each side of the rotating cube.
- Send normal of each vertex to the vertex shader as a vertex attribute.
- The normals must then be rotated in vertex shader before we can use the reflect function to compute the direction of reflection.
- It computes the reflection vector in eye coordinates as a varying variable.
- We assume that the rotation to the cube is applied in the application and its effect is incorporated in the model-view matrix.

Environment Map

Using the reflection vector to locate the corresponding texel in cube map



reflect — calculate the reflection direction for an incident vector

Declaration

genType **reflect**(genType I , genType N);

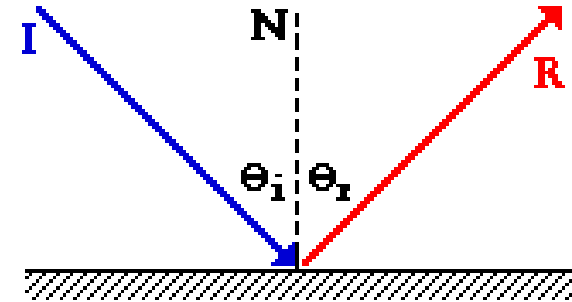
Parameters

I

Specifies the incident vector.

N

Specifies the normal vector.



$$\begin{aligned} \mathbf{R} &= -\mathbf{I} - 2 * (-\mathbf{I}_{\perp}) \\ &= -\mathbf{I} - 2 * (-\mathbf{I} - (-\mathbf{I} \cdot \mathbf{N}) * \mathbf{N}) \\ &= -\mathbf{I} + 2 * (\mathbf{I} + (-\mathbf{I} \cdot \mathbf{N}) * \mathbf{N}) \\ &= -\mathbf{I} + 2 * \mathbf{I} + 2 * (-\mathbf{I} \cdot \mathbf{N}) * \mathbf{N} \\ &= \mathbf{I} + 2 * (-\mathbf{I} \cdot \mathbf{N}) * \mathbf{N} \\ &= \mathbf{I} - 2 * (\mathbf{I} \cdot \mathbf{N}) * \mathbf{N} \end{aligned}$$

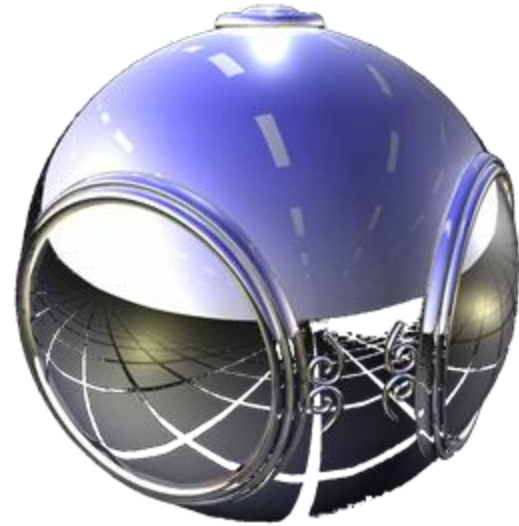
Description

For a given incident vector I and surface normal N , **reflect** returns the reflection direction calculated as $I - 2.0 * \mathbf{dot}(N, I) * N$.

N should be normalized in order to achieve the desired result.

Reflection Map Fragment Shader

```
uniform samplerCube texMap;  
in vec3 R;  
out vec3 FragColor;  
  
void main(void)  
{  
    FragColor = textureCube(texMap, R);  
}
```



Reflection Mapping

- <https://webglsfundamentals.org/webgl/lessons/webgl-environment-maps.html>
- <https://webglsfundamentals.org/webgl/lessons/webgl-skybox.html>
- <http://math.hws.edu/graphicsbook/source/webgl/cube-camera.html>

Toon Shading

- Probably the simplest non-photorealistic shader we can write.
- Uses few color tones, **changes abruptly from tone to tone**, yet it provides a sense of 3D to the model.



- The tones in the teapot above are selected **based on the cosine of the angle** between the light's direction and the normal of the surface.
- >> So if the normal is close to the light's direction, use the brightest tone.
 - As the angle between the normal and the light's direction increases, darker tones are used.
- In other words, the cosine of the angle provides the intensity measure for the tone.

Toon Shading Vertex Shader

```
uniform mat4 ModelView, Projection, NormalMatrix;
uniform vec4 LightPosition;
in vec4 vPosition, normal;
out vec3 normal, lightDir;

void main()
{
    lightDir = normalize(LightPosition.xyz);
    normal = normalize((NormalMatrix*normal).xyz);

    gl_Position = Projection*ModelView*vPosition;
}
```

Toon Shading Fragment Shader

```
precision medium float;
in vec3 normal, lightDir;
out vec4 outcolor;
void main() {
    float intensity;
    vec3 n = normalize(normal);
    vec4 color;
    intensity = max(dot(lightDir, n), 0.0);

    if (intensity > 0.98)
        color = vec4(0.8, 0.8, 0.8, 1.0);
    else if (intensity > 0.5)
        color = vec4(0.4, 0.4, 0.8, 1.0);
    else if (intensity > 0.25)
        color = vec4(0.2, 0.2, 0.4, 1.0);
    else
        color = vec4(0.15, 0.1, 0.1, 1.0);

    outcolor = color;
}
```



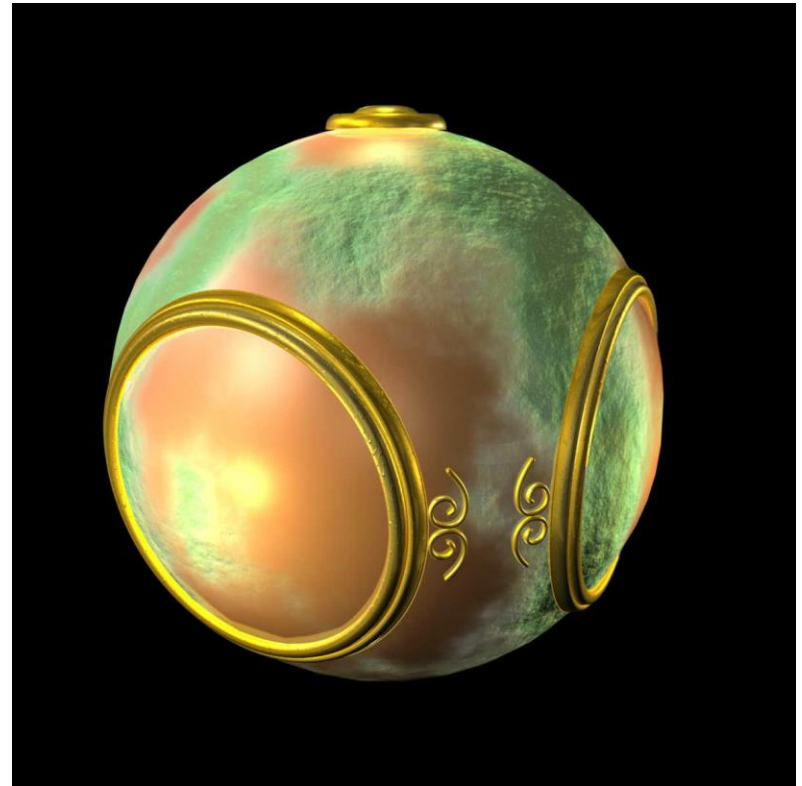
Silhouette?



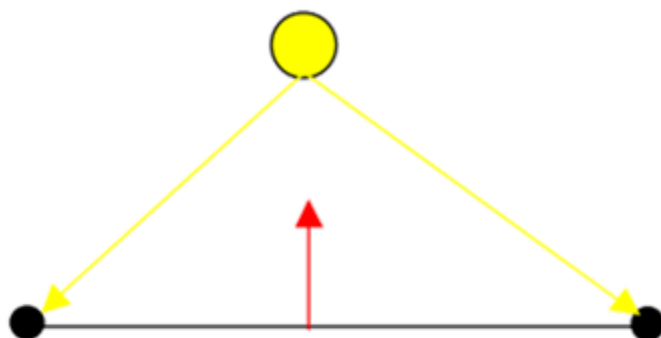
- A simple way is to find the points where $|v \cdot n|$ is small, i.e., where the view vector is almost (or exactly) perpendicular to the surface normal

Bump Mapping

- Solves flatness problem of texture mapping
- Idea:
 - Perturb normal of each fragment
 - Store perturbation as textures

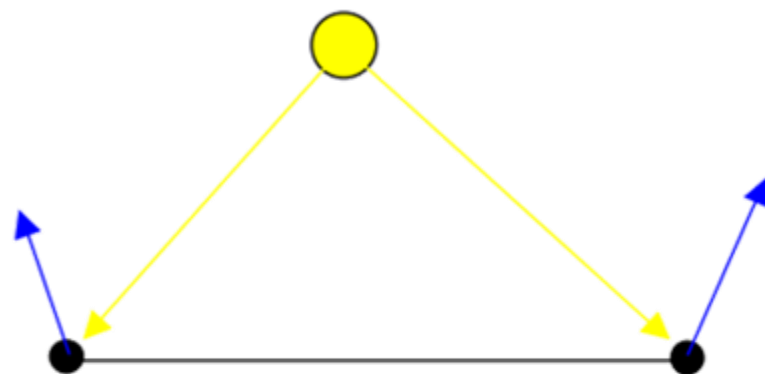


Flat shading



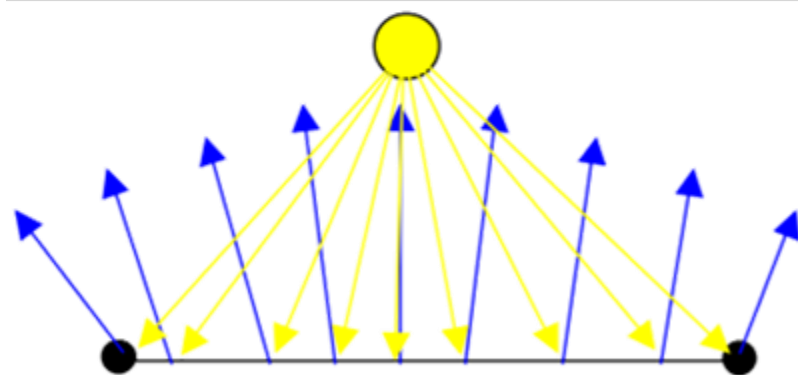
Only the first normal of the triangle is used to compute lighting in the entire triangle.

Gouraud shading



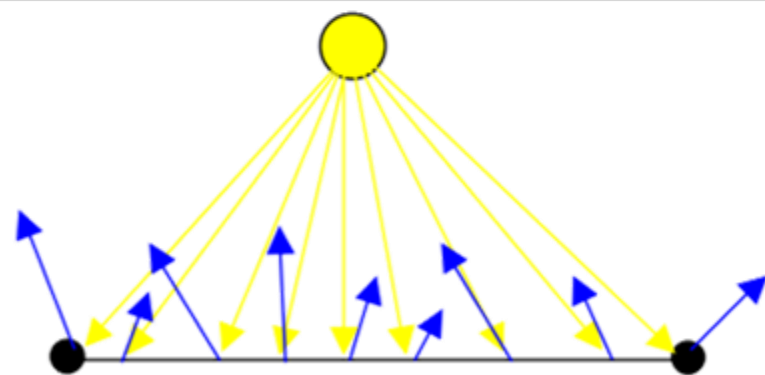
The light intensity is computed at each vertex and interpolated across the surface.

Phong shading



Normals are interpolated across the surface, and the light is computed at each fragment.

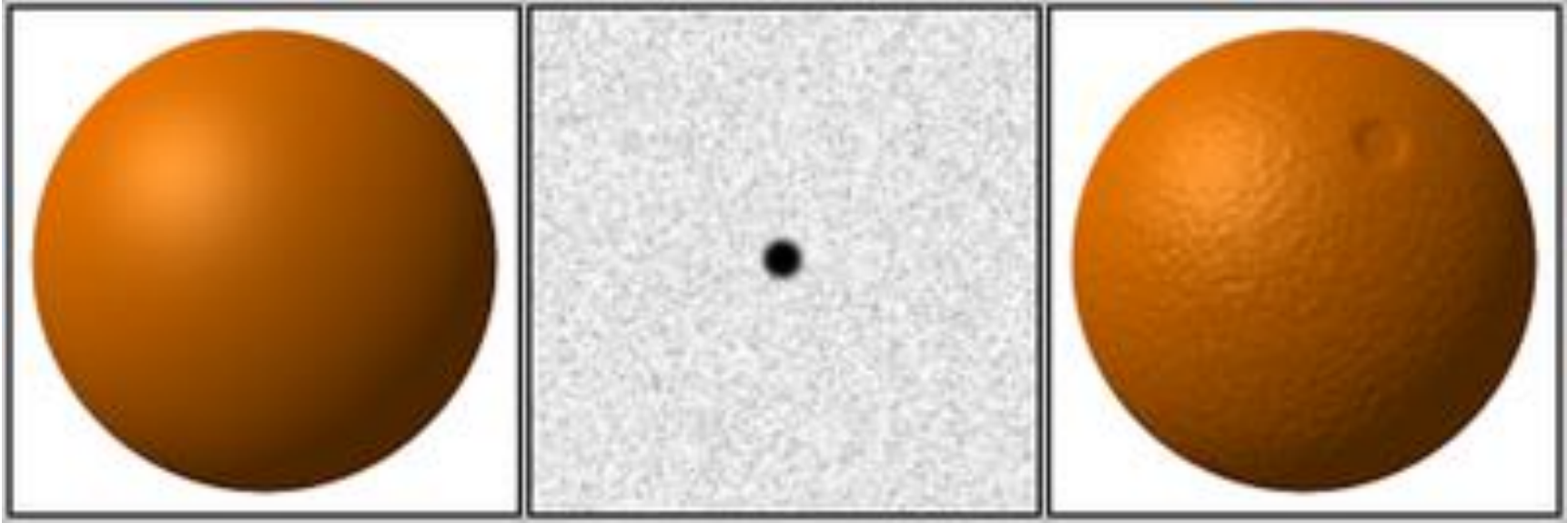
Bump mapping



Normals are stored in a bumpmap texture, and used instead of Phong normals.

Modeling an Orange

- Consider modeling an orange
- Texture map a photo of an orange onto a 3D surface
 - Captures dimples
 - Will not be correct if we move viewer or light
 - We have shades of dimples rather than their correct orientation
- Ideally we need to perturb normals across the surface of the object and compute a new color at each interior point

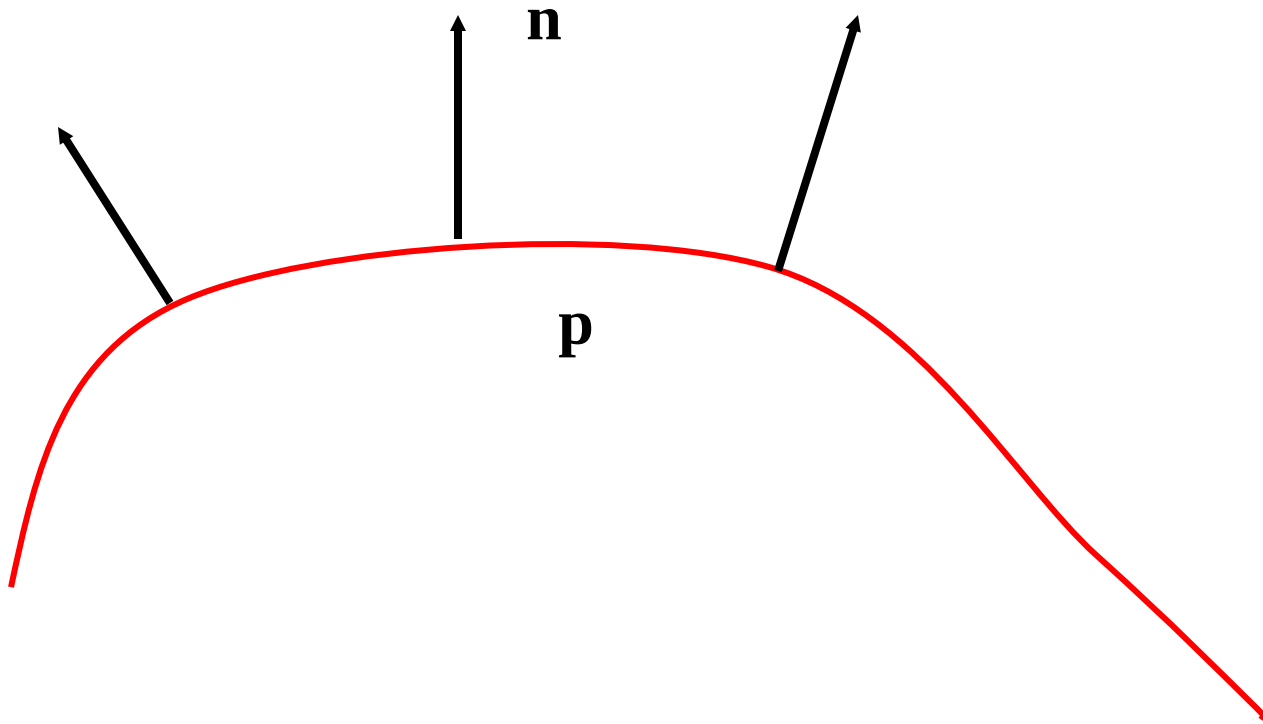


- A sphere without bump mapping (left).
- A bump map to be applied to the sphere (middle).
- The sphere with the bump map applied (right) appears to have a mottled surface resembling an orange.

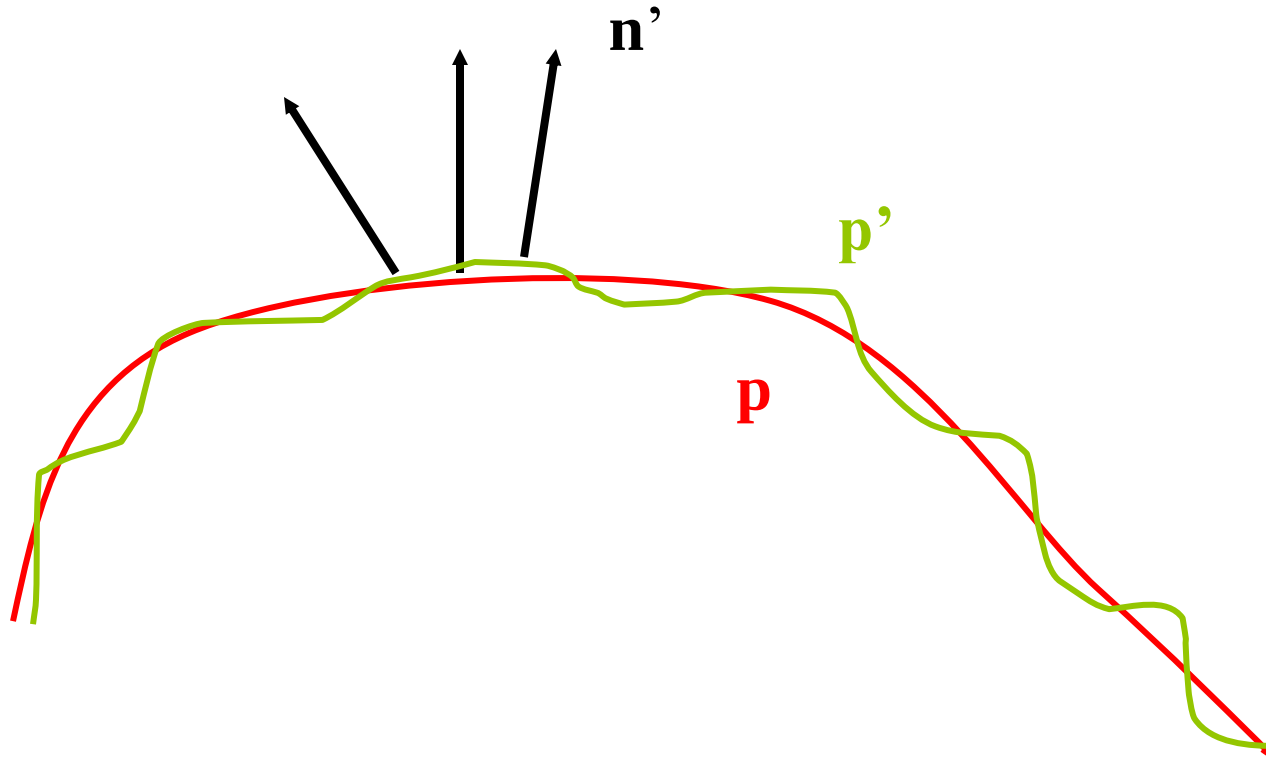
Bump maps achieve this effect by changing how an illuminated surface reacts to light without actually modifying the size or shape of the surface.

Bump Mapping (Blinn)

- Consider a smooth surface



We want to realize this **rougher** look with Bump Mapping



Equations

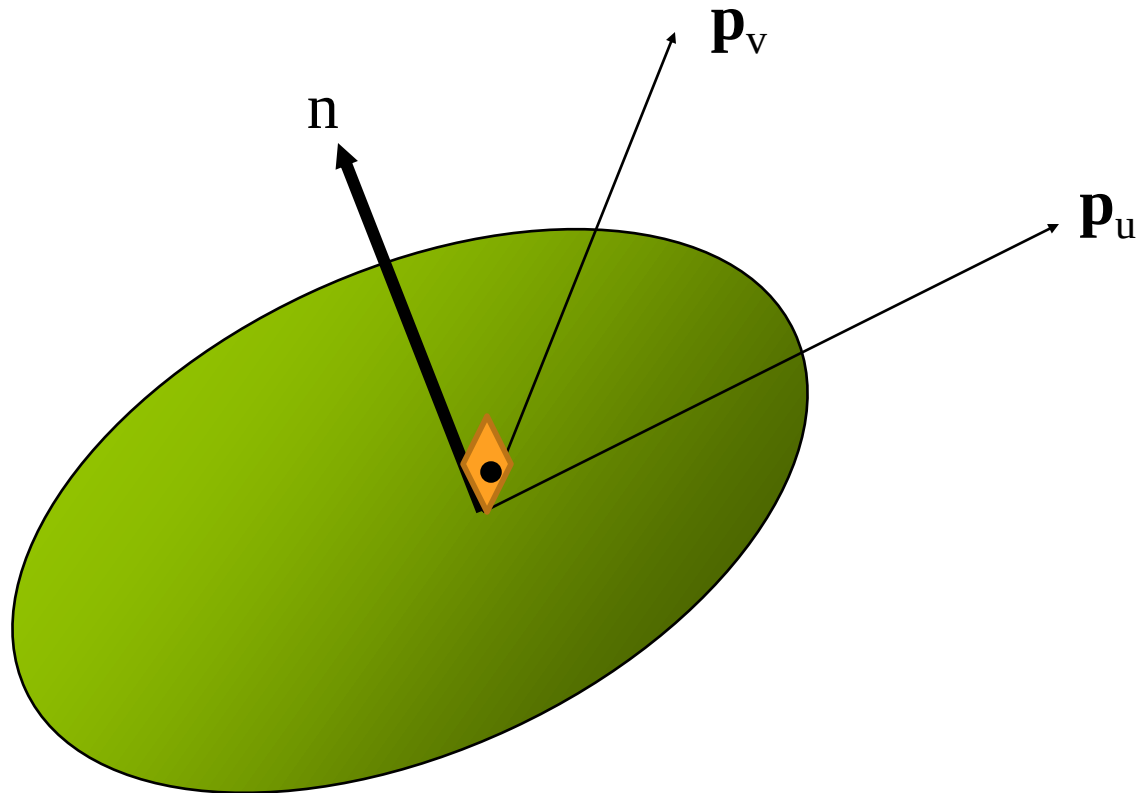
$$\mathbf{p}(u,v) = [x(u,v), y(u,v), z(u,v)]^T$$

$$\mathbf{p}_u = [\partial x / \partial u, \partial y / \partial u, \partial z / \partial u]^T$$

$$\mathbf{p}_v = [\partial x / \partial v, \partial y / \partial v, \partial z / \partial v]^T$$

$$\mathbf{n} = (\mathbf{p}_u \times \mathbf{p}_v) / |\mathbf{p}_u \times \mathbf{p}_v|$$

Tangent Plane



Displacement Function

$$\mathbf{p}' = \mathbf{p} + d(u,v) \mathbf{n}$$

$d(u,v)$ is the bump (displacement) function

$$|d(u,v)| \ll 1$$

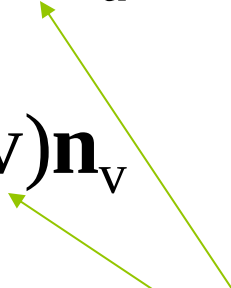
Perturbed Normal

$$\mathbf{n}' = \mathbf{p}'_u \times \mathbf{p}'_v$$

using the product rule

$$\frac{d}{dx} [f(x)g(x)] = f(x)g'(x) + f'(x)g(x)$$

$$\mathbf{p}'_u = \mathbf{p}_u + (\partial d / \partial u) \mathbf{n} + d(u, v) \mathbf{n}_u$$

$$\mathbf{p}'_v = \mathbf{p}_v + (\partial d / \partial v) \mathbf{n} + d(u, v) \mathbf{n}_v$$


since d (displacement) is small, we can neglect last term

Approximating the Normal

$$\mathbf{n}' = \mathbf{p}'_u \times \mathbf{p}'_v$$

$$\approx \mathbf{n} + (\partial \mathbf{d} / \partial u) \mathbf{n} \times \mathbf{p}_v + (\partial \mathbf{d} / \partial v) \mathbf{n} \times \mathbf{p}_u$$

- The vectors $\mathbf{n} \times \mathbf{p}_v$ and $\mathbf{n} \times \mathbf{p}_u$ lie in the tangent plane
- Hence the normal is displaced across the tangent plane
- Must precompute the arrays $\partial \mathbf{d} / \partial u$ and $\partial \mathbf{d} / \partial v$
- Finally, we perturb the normal during shading

Image Processing

- Suppose that we start with a function (bump map) $d(u,v)$
- We can sample it to form a lookup table $D = [d_{ij}]$
- Then $\partial d / \partial u \approx [d_{ij} - d_{i-1,j}]$
and $\partial d / \partial v \approx [d_{ij} - d_{i,j-1}]$
 - We can directly store these values as well
- **Embossing (e.g., Parallax occlusion mapping):**
multipass approach using floating point buffer
 - read more at <https://apoorvaj.io/exploring-bump-mapping-with-webgl/>

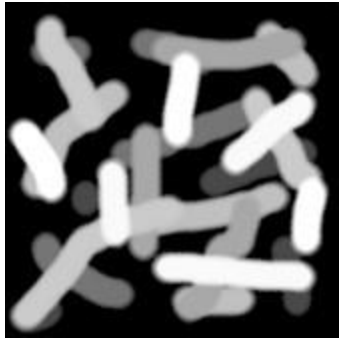
Three scales of detail on an object

- **Macro-features** that cover many pixels
 - Represented by vertices and triangles
- **Micro-features** that are substantially smaller than a pixel
 - Encapsulated in the shading (lighting) model
 - Simulates the light interaction of a surface's microscopic geometry; e.g., shiny objects are microscopically smooth, and diffuse surfaces are microscopically rough
- **Meso-features** that are a few pixels across
 - Describe everything between macro- and micro scales
 - Contains detail that is too complex to render using polygons, but that is large enough for viewers to see changes in curvature over a few pixels
 - Wrinkles on human face

Bump Mapping Family

- A large family of small-scale detail representation techniques
 - Blinn's methods, normal mapping, Parallax mapping, Relief mapping
- By modifying the per-pixel shading routine, give a more 3D appearance than texture mapping, but less than actual geometry manipulation
- Used for meso-scale modeling
 - Adjust shading parameters at the pixel level in such a way that the viewer perceives small perturbation away from the base geometry, which actually remains flat

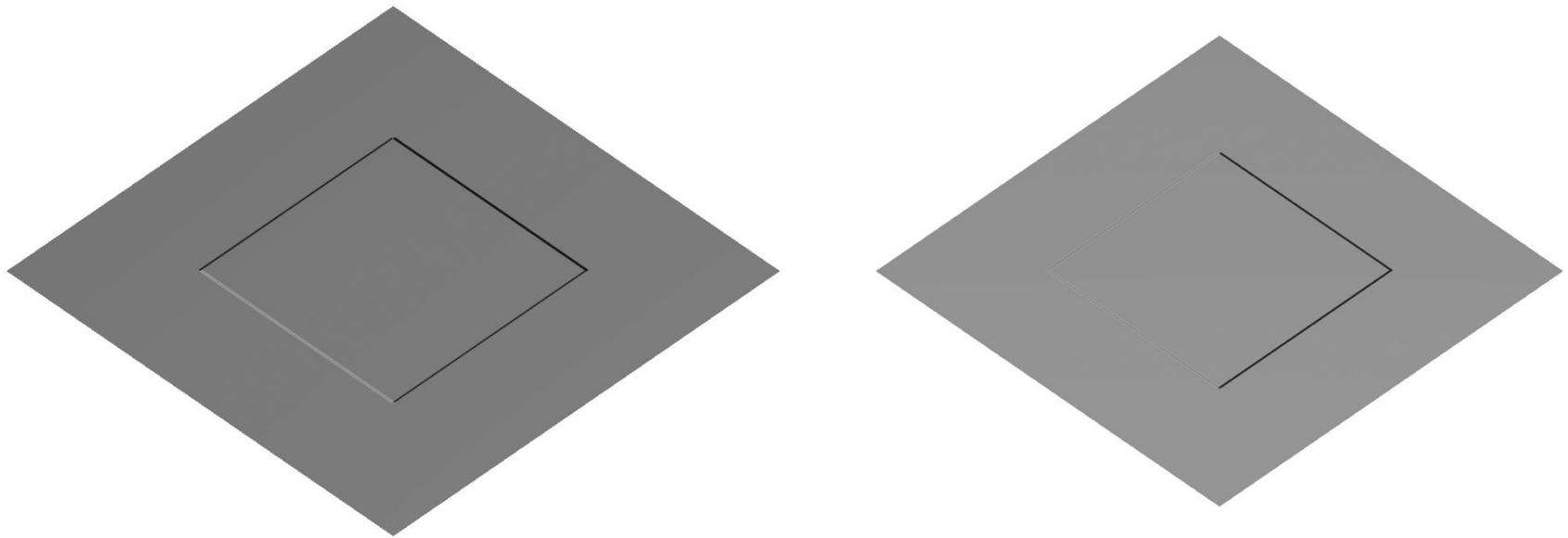
Bump Mapping Example



Bump Mapping Example

<http://www.cs.unm.edu/~angel/WebGL/7E/07/bumpMap.html>

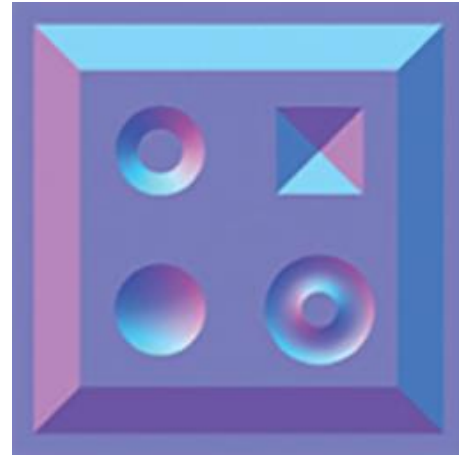
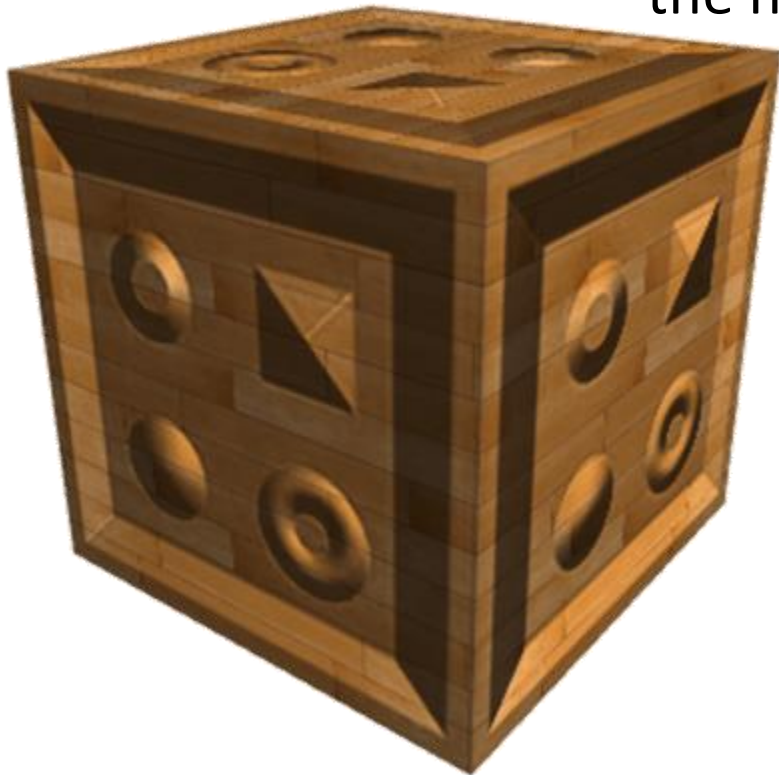
Single Polygon and a Rotating Light Source



Bump Mapping Example

<https://apoorvaj.io/exploring-bump-mapping-with-webgl/>

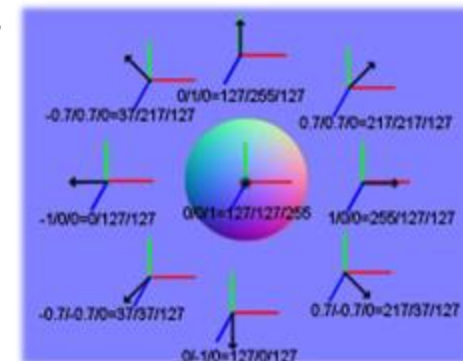
our simple cube is now bump mapped with
the normal map on the right



How to do this?

- The problem is that we want to apply the perturbation at all points on the surface
- Cannot solve by vertex lighting (unless polygons are very small)
- >> Really want to apply to every fragment
 - Can do with a fragment shader

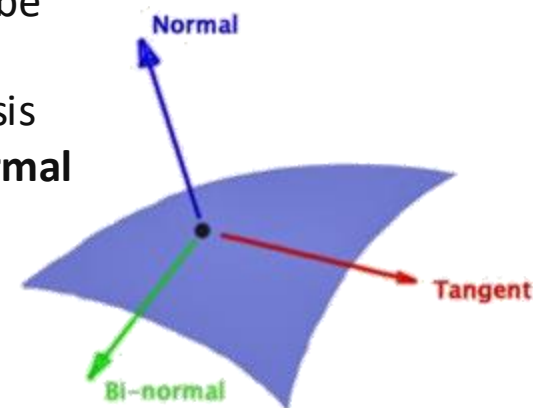
RGB channels are used to encode the normal vector components along the X, Y, and Z axes. Since these can be negative, a channel value of 128 (or 0.5 if normalized) is considered to be 0, e.g. an RGB-encoded (128, 128, 255) denotes a unit vector pointing in the positive Z direction, i.e. the vector $0\hat{i} + 0\hat{j} + 1\hat{k}$.



A problem with this approach is that these surface normals stored in the texture cannot be represented in world space. If they are, then the normals would change every time the normal-mapped object is rotated, and the texture would have to be updated. Moreover, multiple objects with the same meso-features, but with differing geometry and orientations, would not be able to share the same normal map.

>> This is solved by storing them in **tangent space** instead of in world space.

While choosing the basis vectors for the tangent space, we want them to be unperturbed by the changes in orientation of the mesh in world space. In other words, these basis vectors should "stick" to the mesh. The three basis vectors that are chosen are called the **tangent**, the **bitangent**, and the **normal** vectors.

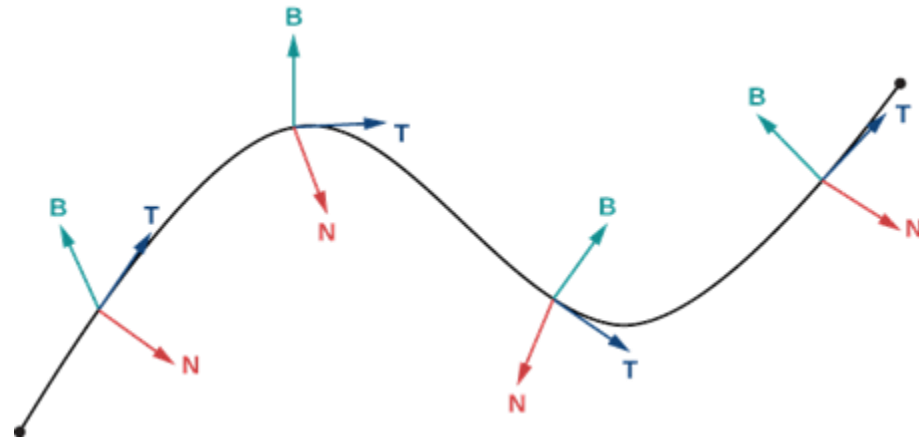


The tangent at each vertex is the partial derivative of the UV texture coordinate with respect to the u component. This means that the tangent points in the direction of change of u at that vertex. The bitangent is similar, but points in the direction of change of the v component.

- These tangents and bitangents are stored in object(model) space - the coordinate space where the object is located at the origin and has unit scale and zero rotation. In the vertex shader, these vectors are transformed into camera space by the normal matrix (inverse transpose of the model-view matrix)
- Once we have the tangent (T_x, T_y, T_z), bitangent (B_x, B_y, B_z) and normal (N_x, N_y, N_z) vectors in world space, we can construct a matrix to change the basis of a vector between the two spaces as needed.

$$V_{tangent\ space} = \begin{pmatrix} T_x & T_y & T_z \\ B_x & B_y & B_z \\ N_x & N_y & N_z \end{pmatrix} \begin{pmatrix} V_x \\ V_y \\ V_z \end{pmatrix}$$

- Our light positions are defined in world space, while our surface normals are in tangent space.
- We need to bring both of these things into the same space to calculate the lighting. Once everything is in the same space, the angle of incidence can be computed along with its cosine, and the Lambertian(diffuse) lighting can be evaluated as usual.



Vertex Shader

```
in vec2 vTexCoord;
```

```
in vec4 vPosition;
```

```
out vec3 L; /* output light vector in texture-space coordinates */
```

```
out vec2 fTexCoord;
```

```
uniform mat4 modelViewMatrix, projectionMatrix;
```

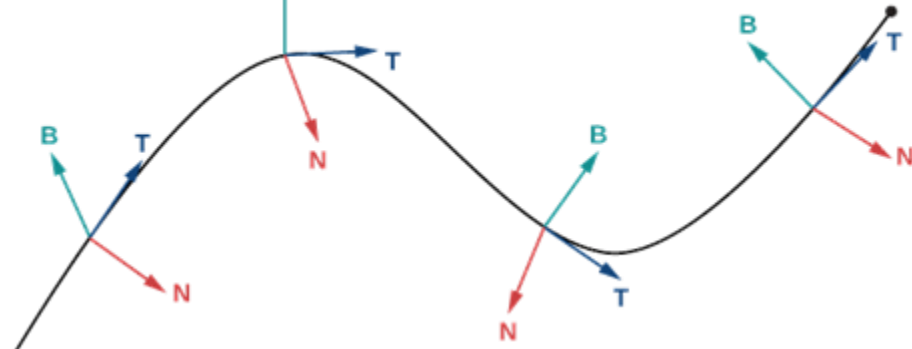
```
uniform mat3 normalMatrix;
```

```
uniform vec4 normal, lightPosition;
```

```
uniform vec3 objTangent; /* tangent vector in object coordinates */
```

Vertex Shader

```
void main() {  
    gl_Position = projectionMatrix*modelViewMatrix*vPosition;  
  
    fTexCoord = vTexCoord;  
  
    /* vertex and light positions brought to eye(camera) space */  
    vec3 eye_vPosition = (modelViewMatrix*vPosition).xyz;  
    vec3 eye_LightPos = (modelViewMatrix*lightPosition).xyz;  
  
    /* normal, tangent and binormal brought to eye(camera) coords */  
    vec3 N = normalize(normalMatrix*normal.xyz);  
    vec3 T = normalize(normalMatrix*objTangent);  
    vec3 B = cross(N, T);
```



Vertex Shader

```
/* compute light vector in eye(camera) space */  
Le = eye_LightPos - eye_vPosition;
```

```
/* bring light vector to texture space */
```

```
L.x = dot(T, Le);
```

```
L.y = dot(B, Le);
```

```
L.z = dot(N, Le);
```

```
L = normalize(L);
```

```
}
```



$$V_{\text{tangent space}} = \begin{pmatrix} T_x & T_y & T_z \\ B_x & B_y & B_z \\ N_x & N_y & N_z \end{pmatrix} \begin{pmatrix} V_x \\ V_y \\ V_z \end{pmatrix}$$

Fragment Shader

```
precision mediump float;
```

← This determines how much precision the GPU uses when calculating floats. Highp is high precision, and more intensive than mediump (medium precision) and lowp (low precision).

```
in vec3 L;
```

```
in vec2 fTexCoord;
```

```
uniform sampler2D texMap; //bumpmap
```

```
uniform vec4 diffuseProduct;
```

```
out fColor;
```

```
void main()
```

```
{
```

```
    vec4 N = texture2D(texMap, fTexCoord);
```

```
    vec3 NN = normalize(2.0*N.xyz - 1.0); //RGB to vector conversion
```

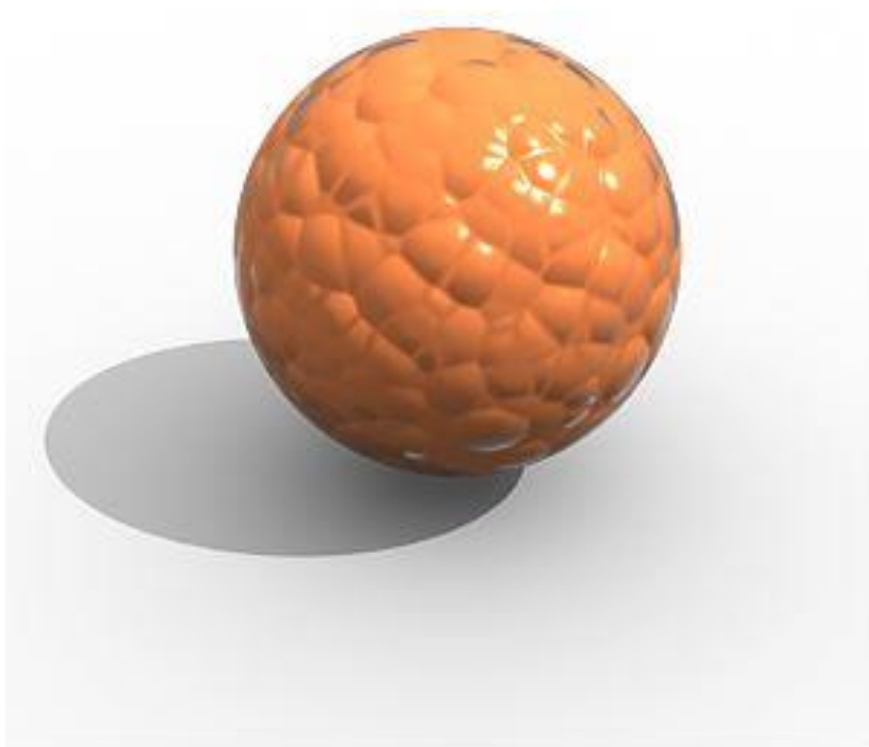
```
    vec3 LL = normalize(L);
```

```
    //compute lighting in texture space per fragment
```

```
    float Kd = max(dot(NN, LL), 0.0);
```

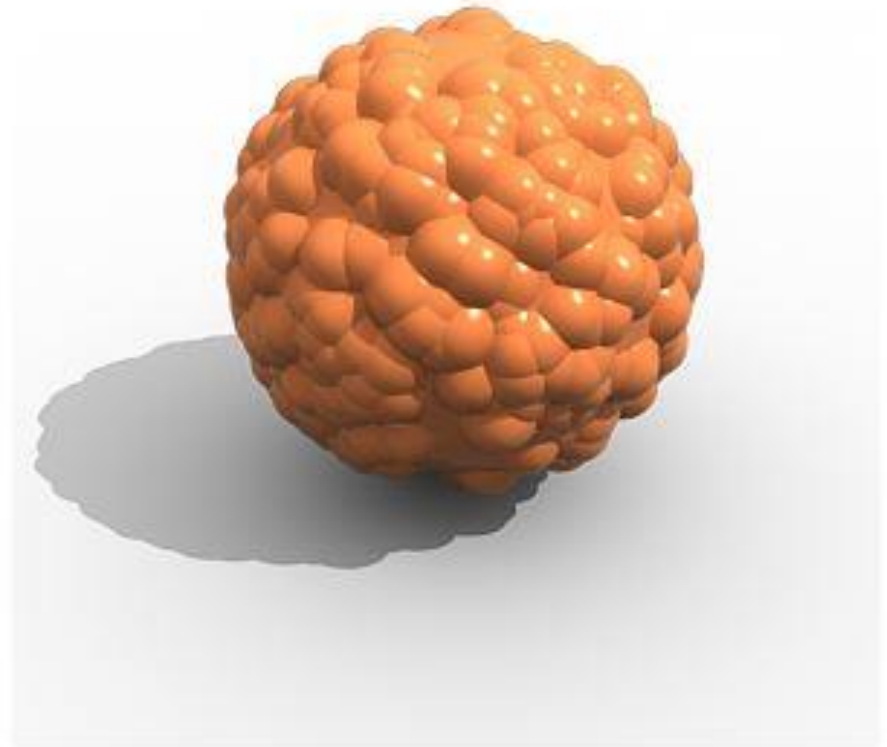
```
    fColor = vec4(Kd*diffuseProduct.xyz, 1.0);
```

```
}
```



Bump mapping

- perturbs normals according to value range in image map
- alters shading calculations to give the illusion of variations in surface geometry



Displacement mapping

- displaces and retessellates geometry along normal (perpendicular to surface) according to value range in image map

Resources

- <http://www.cs.unm.edu/~angel/WebGL/7E/07/>
- <http://www.webglplayground.net/>
- <http://glslsandbox.com/>